

 This work is protected by
US copyright laws and is for
instructors' use only.

Instructor's Manual
to accompany

**The 8051 Microcontroller
and Embedded System**
Using Assembly and C

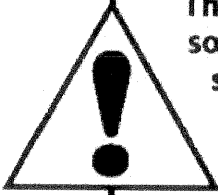
Second Edition

**Muhammad Ali Mazidi
Janice Gillispie Mazidi
Rolin McKinlay**

With Contributions from
Ardeshir Eslami



Upper Saddle River, New Jersey
Columbus, Ohio



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Copyright © 2007 by Pearson Education, Inc., Upper Saddle River, New Jersey 07458.

Pearson Prentice Hall. All rights reserved. Printed in the United States of America. This publication is protected by Copyright and permission should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permission(s), write to: Rights and Permissions Department.

Pearson Prentice Hall™ is a trademark of Pearson Education, Inc.

Pearson® is a registered trademark of Pearson plc

Prentice Hall® is a registered trademark of Pearson Education, Inc.

Instructors of classes using Mazidi, Mazidi, & McKinlay, *The 8051 Microcontroller and Embedded System, Second Edition*, may reproduce material from the instructor's manual for classroom use.



10 9 8 7 6 5 4 3 2 1

ISBN 0-13-119403-8

CONTENTS

Chapter 0	1
Chapter 1	6
Chapter 2	7
Chapter 3	11
Chapter 4	13
Chapter 5	18
Chapter 6	25
Chapter 7	37
Chapter 8	43
Chapter 9	47
Chapter 10	53
Chapter 11	60
Chapter 12	66
Chapter 13	69
Chapter 14	72
Chapter 15	79
Chapter 16	83
Chapter 17	89

CHAPTER 0: INTRODUCTION TO COMPUTING

Section 0.1: Numbering and Coding Systems

1.
 - (a) $12_{10} = 1100_2$
 - (b) $123_{10} = 0111\ 1011_2$
 - (c) $63_{10} = 0011\ 1111_2$
 - (d) $128_{10} = 1000\ 0000_2$
 - (e) $1000_{10} = 0011\ 1110\ 1000_2$
2.
 - (a) $100100_2 = 36_{10}$
 - (b) $1000001_2 = 65_{10}$
 - (c) $11101_2 = 29_{10}$
 - (d) $1010_2 = 10_{10}$
 - (e) $00100010_2 = 34_{10}$
3.
 - (a) $100100_2 = 24_{16}$
 - (b) $1000001_2 = 41_{16}$
 - (c) $11101_2 = 1D_{16}$
 - (d) $1010_2 = 0A_{16}$
 - (e) $00100010_2 = 22_{16}$
4.
 - (a) $2B9_{16} = 0011\ 1011\ 1001_2, 697_{10}$
 - (b) $F44_{16} = 1111\ 0100\ 0100_2, 3908_{10}$
 - (c) $912_{16} = 1001\ 0001\ 0010_2, 2322_{10}$
 - (d) $2B_{16} = 0010\ 1011_2, 43_{10}$
 - (e) $FFFF_{16} = 1111\ 1111\ 1111\ 1111_2, 65535_{10}$
5.
 - (a) $12_{10} = 0C_{16}$
 - (b) $123_{10} = 7B_{16}$
 - (c) $63_{10} = 3F_{16}$
 - (d) $128_{10} = 80_{16}$
 - (e) $1000_{10} = 3E8_{16}$
6.
 - (a) $1001010 = 0011\ 0110$
 - (b) $111001 = 0000\ 0111$
 - (c) $10000010 = 0111\ 1110$
 - (d) $111110001 = 0000\ 1111$
7.
 - (a) $2C+3F = 6B$
 - (b) $F34+5D6 = 150A$
 - (c) $20000+12FF = 212FF$
 - (d) $FFFF+2222 = 12221$

8. (a) $24F-129 = 126_{16}$
 (b) $FE9-5CC = A1D_{16}$
 (c) $2FFFF-FFFF = 30000_{16}$
 (d) $9FF25-4DD99 = 5218C_{16}$
9. (a) Hex: 30, 31, 32, 33, 34, 35, 36, 37, 38, 39
 (b) Binary: 11 0000, 11 0001, 11 0010, 11 0011, 11 0100, 11 0101, 11 0110, 11 0111, 11 1000, 11 1001.

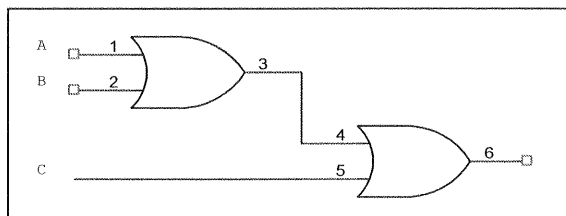
	ASCII (hex)	Binary
0	30	011 0000
1	31	011 0001
2	32	011 0010
3	33	011 0011
4	34	011 0100
5	35	011 0101
6	36	011 0110
7	37	011 0111
8	38	011 1000
9	39	011 1001

10. 000000 22 55 2E 53 2E 41 2E 20 69 73 20 61 20 63 6F 75
 000010 6E 74 72 79 22 0D 0A 22 69 6E 20 4E 6F 72 74 68
 000020 20 41 6D 65 72 69 63 61 22 0D 0A

"U.S.A. is a country".."in North America"..

Section 0.2: Digital Primer

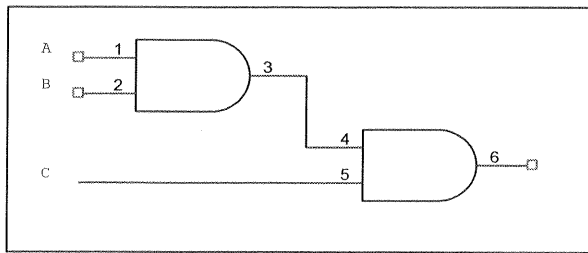
11.



12.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

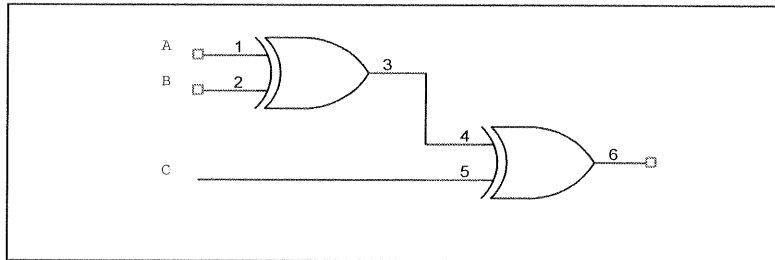
13.



14.

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

15.



A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

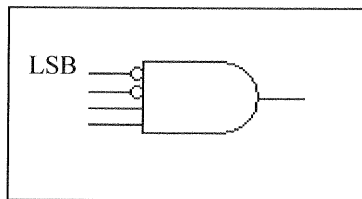
16.

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

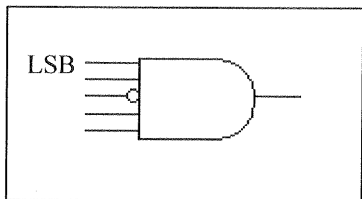
17.

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

18.



19.



20.

CLK	D	Q
No	X	NC
Yes	0	0
Yes	1	1

Section 0.3: Inside the Computer

21. (a) 4
 (b) 4
 (c) 4
 (d) 1 048 576, 2^{20}
 (e) 1024K
 (f) 1 073 741 824, 2^{30}
 (g) 1 048 576 K
 (h) 1024M
 (i) 8388608, 8192K

- 22. 1 million pages
- 23. (a) 589824 bytes
(b) 576 kbytes
- 24. $2^{32} - 1 = 4294967295$
- 25. (a) FFh, 255
(b) FFFFh, 65535
(c) FFFF FFFFh, 4 294 967 295
(d) FFFF FFFF FFFF FFFFh, 18 446 744 073 709 551 615
- 26. (a) 64K
(b) 16M
(c) 4096 Mega, 4G
(d) 256 Tera, 262144 Giga, 268435456 Mega
- 27. Data bus is bidirectional, address is unidirectional.
- 28. PC (Program Counter)
- 29. ALU
- 30. Address, control and data

CHAPTER 1: THE 8051 MICROCONTROLLERS

Section 1.1: Microcontrollers and Embedded Processors

1. False: A general-purpose microprocessor does not have on-chip ROM.
2. True: A microcontroller has on-chip ROM.
3. True: A microcontroller has on-chip I/O ports.
4. True: A microcontroller has a fixed amount of RAM on the chip.
5. CPU, RAM, ROM, I/O, Timer, Serial COM port
6. RAM and ROM
7. Keyboard, mouse, printer
8. Computing power and compatibility with millions and millions of PCs
9. 8051 - Intel, 6811 – Freescale (Motorola), Z8 – Zilog, PIC 16x – Microchip Technology
10. 8051
11. Power consumption
12. The ROM area is where the executable code is stored
13. Very, in case there is a shortage by one supplier
14. Suppliers other than the manufacturer of the chip
15. Only A is true, 8 bit software will run on a 16 bit system

Section 1.1: Overview of the 8051 Family

16. 4096 bytes
17. 128 bytes
18. 2 timers
19. 256 bytes
20. 8031
21. 32 pins
22. 1 serial port
23. UV-EPROM
24. EEPROM
25. NV-RAM
26. Flash
27. (a) 4K ROM, 128 Bytes of RAM
(b) 16K ROM, 256 Bytes RAM
(c) 32K ROM, 256 Bytes RAM
28. The OTP version of the 8051
29. The 8031 does not have on-chip ROM.
30. DS89C420/430 are the best for home development, because you can program it serially using IBM PC.

CHAPTER 2: 8051 ASSEMBLY LANGUAGE PROGRAMMING

Section 2.1: Inside the 8051

1. 8
2. 8
3. 8
4. DPTR or PC (program counter)
5. Necessary
6. 28H, A
7. (a), (d), (f), and (g)
8. (a), (c), (d), (f), and (g)
9. 44H, A
10. 1EH, A

Section 2.2: Introduction to Assembly Programming and

Section 2.3: Assembling and Running an 8051 Program

11. Low, high
12. Assembly language
13. Assembler program
14. True
15. List file
16. False
17. True
18. No
19. They are not real code (real code produces op code). Pseudo-instructions only give instruction to the assembler and does not generate opcodes.
20. True

Section 2.4: The Program Counter and ROM Space in the 8051

21. 0000H
22. It executes whatever is at location 0000h which could be garbage in this case.
23.
 - (a) 2 bytes
 - (b) 2 bytes
 - (c) 1 byte
 - (d) 2 bytes
 - (e) 1 byte
 - (f) 1 byte
 - (g) 1 byte

24.

0000		ORG 0h
0000	7D25	MOV R5, #25h
0002	7F34	MOV R7, #34h
0004	7400	MOV A, #0
0006	2D	ADD A, R5
0007	2F	ADD A, R7
0008	2412	ADD A, #12h
000A	80FE	HERE: SJMP HERE
000C		END

25. (a) 3FFFh (16K)
(b) 1FFFh (8K)
(c) 7FFFh (32K)
(d) 1FFFh (8K)
(e) 0FFFh (4K)
(f) 0FFFh (4K)
(g) FFFFh (64K)

26. 0000h, FFFFh
27. 32K
28. 1K

Section 2.5: 8051 Data Types and Directives

29.

0200		1	ORG 200H	
0200	45617274	2	MYDATA_1:	DB "Earth"
0204	68			
0205	3938372D	3	MYDATA_2:	DB "987-65"
0209	3635			
020B	47414245	4	MYDATA_3:	DB "GABEH 98"
020F	48203938			
		5		
		6	END	

30.

0340		1	ORG 340H	
0340	16569920	2	DAT_1:	DB 22, 56H, 10011001B, 32, 0F6H, 11111011B
0344	F6FB			
		3	END	

All in hex where contents of each location are:

340=(22), 341=(56), 342=(99), 343=(20), 344=(F6), 345=(FB)

Section 2.6: 8051 Flag Bits and the PSW Register

31. 8
32. D7, D6

33. D2, D0
34. when there is a carry from D7
35. when there is a carry from D3 to D4
36. CY = 1
37. (a) CY = 1
(b) CY = 0
(c) CY = 0
- 38.

```

ORG 0H
MOV R0, #5
MOV A, #0
HERE:
    ADD A, #55h
    DJNZ R0, HERE
END

```

Section 2.7: 8051 Register Banks and Stack

39. D3, D4
40. 08
41. Bank 1
42. 8 Bits
43. Bank 0
44. Bank 0:00h-07h, Bank 1: 08h-0Fh, Bank 2: 10h-17h, Bank 3: 18h-1Fh
45. (a) RAM Location 04
(b) RAM Location 00
(c) RAM Location 07
(d) RAM Location 05
46. (a) RAM Location 14h
(b) RAM Location 10h
(c) RAM Location 17h
(d) RAM Location 15h
47. SETB PSW.4
- 48.

INSTRUCTION	SP after execution of the instruction	Contents of the Stack
ORG 0	07H	?
MOV R0, #66H	07H	?
MOV R3, #7FH	07H	66H
MOV R7, #5DH	08H	7FH
PUSH 0	09H	5DH
PUSH 3	0AH	5DH
PUSH 7	0AH	5DH
CLR A	0AH	5DH
MOV R3, A	0AH	5DH
MOV R7, A	0AH	5DH

POP 3	09H	7FH
POP 7	08H	66H
POP 0	07H	7FH

49. NO, pop 7
pop 3
pop 0

50.

INSTRUCTION	SP after execution of the instruction	Contents of the Stack
ORG 0	07H	?
MOV SP, #70H	70H	?
MOV R5, #66H	70H	?
MOV R2, #7FH	70H	?
MOV R7, #5DH	70H	?
PUSH 5	71H	66H
PUSH 2	72H	7FH
PUSH 7	73H	5DH
CLR A	73H	5DH
MOV R2, A	73H	5DH
MOV R7, A	73H	5DH
POP 7	72H	7FH
POP 2	71H	66H
POP 5	70H	?

CHAPTER 3: JUMP, LOOP, AND CALL INSTRUCTIONS

Section 3.1: Loop and Jump Instructions

1. 256
2. the instruction following the jump
3. PC
4. short jump, 2
5. long jump, 3
6. less ROM space is consumed
7. True
8. False
9. c, LJMP is not a short jump.
10. 2, because it can only go -128 to 127 bytes in the program
11. True
- 12.

```
MOV R6, #10
BACK: MOV R5, #100
HERE: DJNZ R5, HERE
      DJNZ R6, BACK
```

13.

```
MOV R6, #10
BACK: MOV R5, #100
AGAIN: MOV R4, #100
HERE: DJNZ R4, HERE
      DJNZ R5, AGAIN
      DJNZ R6, BACK
```

14. 20,000
15. 128
16. 127

Section 3.2: Call Instructions

17. 3
18. 2
19. 2K
20. 64K
21. 2 bytes
22. 2 bytes
23. They need to be equal (in order for the stack to remain unchanged) so that when the RET instruction is executed it goes back to place where it originated; otherwise, it returns to a wrong place.

24. When POP is executed, the contents of the stack are copied to the destination register and SP is decremented by one.
25. SP Stack content
 09 (00)
 08 (0E)
26. SP Stack content
 0B (67)
 0A (99)
 09 (02)
 08 (0B)

Section 3.3: Time Delay Generation for Various 8051 Chips

27. 10 MHz
28. $0.667 \mu s$
29. $1 \mu s$
30. $0.480 \mu s$
31. True, both instructions take 2 machine cycles to execute.
32. $813.75 \mu s$
33. $750 \mu s$
34. $((4 \times 250) \times (200 \times 100)) \times 1.085 \mu s = 21.7 \text{ seconds}$
35. $(250 \times 150) \times 5 \times 0.75 \mu s = 140625 \mu s$
36. Duration of each machine cycle: $1/(11.0592\text{MHz}) = 90.422 \text{ ns}$
 Number of machine cycles: $(1 + 1 + 1 + 4) \times 150 \times 90.422\text{ns} = 94.944 \mu s$
37. $1 / 16\text{MHz} = 62.5 \text{ ns}$
 $(1+1+1+4) \times 200 \times 62.5 \text{ ns} = 87.5 \mu s$
38. $(1+1+4) \times 250 \times 200 \times 100 \times 90.422 \text{ ns} = 2.713 \text{ seconds}$
39. $(7 \times 250) \times 150 \times 62.5 \text{ ns} = 16.406 \text{ ms}$
40. Although the period of each machine cycle is twelve times shorter in DS89C420/30, the fact that the execution of many instructions take more number of machine cycles than in AT89C51, does not allow the performance to become twelve times faster in DS89C420/30. As an example, if you divide the answer of Problem 35 by the answer of Problem 39, we can figure out that in this particular case, DS89C420/30 is almost 8.6 times faster than AT89C51, not 12 times faster.

CHAPTER 4: I/O PORT PROGRAMMING

Section 4.1: 8051 I/O Programming

1. 40
2. Pins 40, 20 are assigned to Vcc and Gnd, respectively.
3. 32
4. 8 Pins, 39-32, are designated for P0.
5. 8 Pins, 01-08, are designated for P1.
6. 8 Pins, 21-28, are designated for P2.
7. 8 Pins, 10-17, are designated for P3.
8. output
9. P0
10. P1
- 11.

```
ORG 0000h
MOV A, #0FFH
MOV P1, A

MOV A, P1
MOV P0, A
MOV P2, A
MOV P3, A
HERE: SJMP HERE
END
```

12.

```
ORG 0000h
MOV A, #0FFH
MOV P2, A

MOV A, P2
MOV P0, A
MOV P1, A
HERE: SJMP HERE
END
```

13. 10 and 11
14. 0000h; This means that the starting of the code must begin at location 0000h. In other words it belongs to code ROM.

15. (a)

```
ORG 0000h

AGAIN:
    MOV A, #55h
    MOV P1, A
    MOV P2, A
    ACALL DELAY
    MOV A, #0AAh
    MOV P1, A
    MOV P2, A
    ACALL DELAY
    SJMP AGAIN

END
```

(b)

```
ORG 0000h

    MOV A, #55h
AGAIN:
    MOV P1, A
    MOV P2, A
    ACALL DELAY
    CPL A
    SJMP AGAIN

END
```

Section 4.2: I/O Bit Manipulation Programming

- 16. All ports of the 8051 are bit addressable.
- 17. The advantage of the bit addressing is that it allows each bit to be modified without affecting the other bits.
- 18. P1.x
- 19. No, CPL P1 is not a valid instruction because CPL works with register A and individual bits.
- 20.

```
ORG 0000h

AGAIN:
    SETB P1.2
    SETB P1.5
    ACALL DELAY
    CLR P1.2
    CLR P1.5
    ACALL DELAY
    SJMP AGAIN

END
```

21.

```
ORG 0000h

AGAIN:
    SETB P1.3
    SETB P1.7
    SETB P2.5
    ACALL DELAY
    CLR P1.3
    CLR P1.7
    CLR P2.5
    ACALL DELAY

    SJMP AGAIN

END
```

22.

```
ORG 0000h

    SETB P1.3
AGAIN:
    JNB P1.3, AGAIN
    MOV A, #55h
    MOV P2, A

    HERE: SJMP HERE

END
```

23.

```
ORG 0000h

    SETB P1.3
AGAIN:
    JB P1.3, AGAIN
HERE:
    MOV A, #55h
    MOV P0, A
    ACALL DELAY
    MOV A, #0AAh
    MOV P0, A
    ACALL DELAY

    SJMP HERE

END
```

24.

```
ORG 0000h

SETB P2.0
AGAIN:
    JB  P2.0, HERE
    MOV A, #66H
    MOV P1, A
    SJMP AGAIN
HERE:
    MOV A, #99H
    MOV P1, A
    SJMP AGAIN
END
```

25.

```
ORG 0000h

SETB P1.5
AGAIN:
    JNB P1.5, AGAIN

    CLR P1.3
    SETB P1.3
    CLR P1.3

HERE: SJMP HERE
END
```

26.

```
ORG 0000h

SETB P1.3
AGAIN:
    JB  P1.3, HERE

    CLR P1.4
    SJMP AGAIN

HERE: SETB P1.4
    SJMP AGAIN
END
```

27. The fifth bit of port 1.

28.

```
ORG 0000H

SETB P1.7
SETB P1.6

AGAIN:
    JB P1.7, HERE
    CLR P1.0
    SJMP NEXT

HERE:
    SETB P1.0

NEXT:
    JB P1.6, COPY
    CLR P1.7
    SJMP AGAIN

COPY:
    SETB P1.7
    SJMP AGAIN
END
```

CHAPTER 5: 8051 ADDRESSING MODES

Section 5.1: Immediate and Register Addressing Modes

Section 5.2: Accessing Memory using Various Addressing Modes

1. (b)
2. (a) immediate (g) direct addressing
(b) direct addressing (h) direct addressing
(c) direct addressing (i) register indirect
(d) immediate (j) direct addressing
(e) direct addressing (k) register indirect
(f) immediate
3. (a) 00h (g) 0Ch
(b) 0E0h (h) 82h
(c) 07h (i) 0Eh
(d) 13h (j) 18h
(e) 0F0h (k) 83h
(f) 1Fh (l) 80h
4. Bank 1
5. direct
6. “MOV A, 0F0h” moves the value stored in the B register into register A.
7. “MOV A, 1Fh” moves the value of R7 of bank 3 into register A.
- 8.

```
PUSH 00h
PUSH 01h
PUSH 03h

POP 1Fh
POP 1Eh
POP 1Dh
```

9. R0 and R1
- 10.

```
MOV A, #0FFh
MOV R1, #50h
MOV R7, #20h
AGAIN:
MOV @R1, A
INC R1
DJNZ R7, AGAIN
```

11.

```

    ORG 0000h

    MOV DPTR, #400h

    MOV R7, #10
    MOV R1, #30h
AGAIN:
    CLR A
    MOVC A, @A+DPTR
    MOV @R1, A
    INC R1
    INC DPTR
    DJNZ R7, AGAIN

    END

```

12.

```

    ORG 0000h

    MOV DPTR, #250h
    MOV A, #0FFh
    MOV P1, A
AGAIN:
    MOV A, P1
    MOVC A, @A+DPTR
    MOV P2, A
    SJMP AGAIN

    ORG 250h
TABLE: DB 5,8,13,20,29,40,53,68,85,104
    END

```

13.

```

    ORG 0000h

    MOV DPTR, #250h
    CLR A
    MOV 30H, A
    MOV R0, #5
AGAIN:
    CLR A
    MOVC A, @A+DPTR
    INC DPTR
    ADD 30H, A
    DJNZ AGAIN

    HERE: SJMP HERE

    ORG 250h
TABLE: DB 06,09,02,05,07
    END

```

Section 5.3: Bit Addresses for I/O and RAM

- 14. invalid
- 15. valid
- 16. valid
- 17. All of the ports
- 18. B, PSW, IP, IE, ACC, TCON, SCON (and all the ports P0 - P3)
- 19. (a) Invalid (e) D4 of B
(b) D3 of P2 (f) P0.0
(c) D5 of A (g) D3 of PSW
(d) P1.0 (h) P0.7

20.

HERE:

```
SETB P1.5
LCALL DELAY
LCALL DELAY
LCALL DELAY
CLR P1.5
LCALL DELAY
SJMP HERE
```

21.

HERE:

```
SETB P2.7
LCALL DELAY
LCALL DELAY
LCALL DELAY
LCALL DELAY
CLR P2.7
LCALL DELAY
SJMP HERE
```

22.

HERE:

```
CLR P2.7
SETB P1.4
JNB P1.4, HERE

SETB P2.7
LCALL DELAY
CLR P2.7
LCALL DELAY
SJMP HERE
```

23.

HERE:

```
SETB P2.1
JB P2.1, HERE
MOV P0, #55H
```

24. 80h – 87h
25. 90h – 97h
26. A0h – A7h
27. B0h – B7h
28. None
29. 88h – 8Fh
30. E0h – E7h
31. F0h – F7h
32. D0h – D7h
33. (a) P0.5 (g) P2.7
(b) P0.7 (h) P3.3
(c) TCON.0 (i) PSW.4
(d) TCON.5 (j) PSW.7
(e) P1.3 (k) D3 of B
(f) P2.5

34.

CLR PSW.3 SETB PSW.4 MOV R3, A MOV R5, B

35. CLR PSW.7
36. The difference between the two, is that the CLR C command is a 1 byte instruction, whereas the CLR PSW.7 is a two byte instruction.
37. JNB PSW.2,...
38. JB PSW.7,...
39. JB PSW.0,...
40. JB PSW.6
41. CY = D7H
P = D0H
AC = D6H
OV = D2H
42. 16, 20h – 2Fh
43. 00h – 7Fh
44. 00h – 7Fh
45. All the SFR are located in the address space of 80h – FFh, but not all of these addresses are used. The unused portion is reserved for future.
46. (a) 00-7Fh (b) b 80-FFh
There are five gaps in the SFR portion. By referring to figure 5.2, these gaps are: C0h~C7h, C8~CFh, D8h~DFh, E8h~EFh, and F8h~FFh. These gaps are reserved for future improvements like T2CON in 8052 which occupies C8h~CFh bit locations.
47. They belong to byte address space of 20h-2Fh as shown below:
(a) D5 of byte 20h (g) D7 of byte 2Ch
(b) D7 of byte 28h (h) D5 of byte 2Ah
(c) D0 of byte 23h (i) D4 of byte 22h

(d) D5 of byte 25h

(e) D3 of byte 2Ah

(f) D5 of byte 22h

(j) D7 of byte 26h

(k) D7 of byte 2Fh

48.

True

49.

True

50.

```
MOV 04, C
```

51.

```
MOV C, PSW.6  
MOV 16H, C
```

52.

```
MOV C, PSW.0  
MOV 12H, C
```

53.

```
JNB ACC.0, OVER  
JNB ACC.1, OVER  
MOV B, #4  
DIV AB  
OVER:
```

54.

```
MOV C, ACC.7  
JNC EXIT  
MOV DPTR, #300h  
LCALL DISPLAY  
  
EXIT: SJMP EXIT  
  
ORG 300h  
DATA1: DB "A Negative Number"
```

55.

```
MOV A, B  
JB ACC.7, OVER  
ACALL DISPLAY  
...  
...  
OVER:
```

56.

```
MOV R1, #10h
MOV A, #0FFH
MOV R0, #20h
KEEP: MOV @R0, A
      INC R0
      DJNZ R1, KEEP
```

```
SETB 00H
SETB 01H
SETB 02H
SETB 03H
SETB 04H
...
and so on.
```

57.

```
JB ACC.0, OVER
JB ACC.1, OVER
JB ACC.2, OVER
MOV B, #8
DIV AB
.....
OVER:
```

58.

```
MOV R4, #0
MOV R3, #8
MOV A, R2
HERE:
      RRC A
      JC, NEXT
      INC R4
NEXT: DJNZ R3, HERE

;NOW R4 Contains the number of zeros
```

Section 5.4: Extra 128-Byte On-Chip RAM in 8052

59. 256 bytes of RAM + 128 SFR bytes = 384 bytes. In 8051, we have only 256 bytes (128 byte SFR + 128 byte RAM).
60. Direct addressing mode.
61. Register indirect addressing mode.
62. Lower 128-bytes: 00h-7Fh
Upper 128-bytes: 80h-FFh
63. Upper
64. Physically they are two different memory spaces, otherwise it would be meaningless to say that there is another 128-byte memory space in 8052. And that is why we need to use two different addressing modes to access each of these memory spaces.
65. (a) Puts 99h in the address 80h of the SFRs (which is P1)
(b) Writes 99h into the address 80h of the upper 128-byte memory space (which is the first byte of the upper part of memory)
66. R1 and R0

67.

```
ORG 0000H
MOV R0,#0C0H
MOV R2,#16
MOV A,#55H
AGAIN:
MOV @R0,A
INC R0
DJNZ R2,AGAIN

SJMP $
END
```

68.

```
ORG 0000H
MOV R0,#60H
MOV R1,#0D0H
MOV R2,#16
AGAIN:
MOV A,@R0
MOV @R1,A
INC R0
INC R1
DJNZ R2,AGAIN

SJMP $
END
```

CHAPTER 6: ARITHMETIC, LOGIC INSTRUCTIONS, AND PROGRAMS

Section 6.1: Arithmetic Instructions

1. (a) CY=0, AC=1
(b) CY=0, AC=1
(c) CY=1, AC=0
(d) CY=1, AC=1
(e) CY=1, AC=0
(f) CY=0, AC=0

2.

```
        ACALL TRANSF
        ACALL ADD_IT
        ...
TRANSF:
        MOV    DPTR, #300H
        MOV    R3, #09H
        MOV    R0, #40H
BACK:   CLR    A
        MOVC   A, @A+DPTR
        MOV    @R0, A
        INC    R0
        INC    DPTR
        DJNZ   R3, BACK
        RET

;-----
ADD_IT:
        CLR    A
        MOV    R3, #9
        MOV    R0, #40H
RE:     ADD    A, @R0
        DA     A
        INC    R0
        DJNZ   R3, RE
        MOV    R3, A
        RET
        ORG    300H
NUMBERS: DB     9, 9, 9, 9, 9, 9, 9, 6, 8, 2
```

3.

```
;The transfer part is the same as in Problem 2
; ADD_IT is modified as follows:
ADD_IT:
    CLR    A
    MOV    B,A
    MOV    R3,#9
    MOV    R0,#40H
RE:    ADD    A,@R0
    JNC    OVER
    INC    B
OVER:
    INC    R0
    DJNZ   R3,RE
    MOV    R2,A
    MOV    R3,B
    RET
```

4.

```
;in the add subroutine we simply put
;the "DA A" instruction

ADD_IT:
    CLR    A
    MOV    B,A
    MOV    R3,#9
    MOV    R0,#40H
RE:    ADD    A,@R0
    DA     A
    JNC    OVER
    INC    B
OVER:
    INC    R0
    DJNZ   R3,RE
    MOV    R2,A
    MOV    R3,B
    RET
```

5.

```

        MOV R3, #16
        MOV R1, #40H
        MOV A, #55H
BACK:   MOV @R1, A
        INC R1
        DJNZ R3, BACK

        MOV R4, #16
        MOV R1, #40H
        CLR A
        MOV B, A
HERE:   ADD A, @R1
        JNC OVER
        INC B
OVER:   INC R1
        DJNZ R4, HERE
        MOV 60H, A
        MOV 61H, B

```

6. (a) Take the 2's complement of 12h. Add the result to 23h. Invert the carry.

23	0010 0011	→	0010 0011
- 12	0001 0010	→ +	1110 1110
			1 0001 0001

where 1→0 for carry

(b) 53 → 0101 0011 → 1010 1101
+ 0100 0011
0 1111 0000 → F0
0→1 for carry

The result (F0h) is negative, which is the 2's complement of 10h

(c) 1001 1001 → 0110 0111
+ 1001 1001
1 0000 0000
1→0 for carry, and the result is zero.

7.

```
; (a) --
    ORG 0H
    CLR C
    MOV A, #23H
    MOV R2, #12H
    SUBB A, R2

; (b) -----
    CLR C
    MOV A, #43H
    MOV R2, #53H
    SUBB A, R2

; (c) -----
    CLR C
    MOV A, #99
    MOV R2, #99
    SUBB A, R2
```

8. True, it must be used after the ADD or ADDC instruction.

9.

```
CLR C
MOV    A, #9AH
ADDC   A, #48H
MOV    40H, A
MOV    A, #7FH
ADDC   A, #0BCH
MOV    41H, A
MOV    A, #19H
ADDC   A, #34H
MOV    42H, A
```

10.

```
CLR C
MOV    A, #48H
SUBB   A, #9AH
MOV    40H, A
MOV    A, #0BC
SUBB   A, #7FH
MOV    41H, A
MOV    A, #34H
SUBB   A, #19H
MOV    42H, A
```

11.

```
CLR C
MOV  A, #95H
ADDC A, #48H
DA   A
MOV  40H, A
MOV  A, #77H
ADDC A, #45H
DA   A
MOV  41H, A
MOV  A, #19H
ADDC A, #34H
DA   A
MOV  42H, A
```

12.

```
MOV A, #77
MOV B, #34
MUL AB
```

13.

```
MOV A, #77
MOV B, #3
DIV AB
```

14. False, it only works with A and B.

15.

```

    ORG    0000
    ACALL TRANSFER
    ACALL ADD_IT
    MOV    70H,A
    ACALL DIVIDE
    SJMP   $

TRANSFER:
    MOV R0, #09
    MOV DPTR, MYDATA
    MOV R1, #30H
HERE:   CLR A
        MOVC A, @A+DPTR
        MOV @R1, A
        INC R1
        INC DPTR
        DJNZ R0, HERE
        RET

;-----
ADD_IT:
    MOV R0, #09
    MOV R1, #30H
    CLR A
HERE2:  ADD A, @R1
        INC R1
        DJNZ R0, HERE2
        RET

;-----
DIVIDE:
    MOV A, 70H
    MOV B, #09
    DIV AB
    MOV R7, A
    RET

    ORG 250H
MYDATA:  DB    3,9,6,9,7,6,4,2,8
    END

```

Section 6.2: Signed Number Concepts and Arithmetic Operations

16. (a) 1110 1001
 (b) 0000 1100
 (c) 1110 0100
 (d) 0110 1111
 (e) 1000 0000
 (f) 0111 1111

17. unsigned

18.

(a) MOV A, #15	(b) MOV A, #-123	(c) MOV A, #25H	(d) MOV A, #-127
MOV R1, #-12	MOV R1, #-127	MOV R1, #34H	MOV R1, #127
ADD A, R1	ADD A, R1	ADD A, R1	ADD A, R1
OV = 0	OV = 1	OV = 0	OV = 0

19. The CY flag is raised when there is carry beyond bit D7 and used for unsigned numbers. The OV is raised when there is a carry from D6 to D7 but not from D7 to carry. Or when there is a carry from D7 to carry but not from D6 to D7 and it is used for signed numbers.
20. The OV flag is raised when there is a carry from D6 to D7 but not from D7 to carry. Or when there is a carry from D7 to carry but not from D6 to D7.
21. PSW.2
22. The OV flag is detected using the instruction “JNB PSW.2,...” The carry flag can be detected by the JNC instruction.

Section 6.3: Logic and Compare Instructions

23.
 - (a) 40h
 - (b) F6h
 - (c) 86h
 - (d) 90h
 - (e) 60h
 - (f) F0h
 - (g) F0h
 - (h) F9h
 - (i) 1Eh
 - (j) 5Ah
24.
 - (a) 64h
 - (b) 7bh
 - (c) 3Fh
 - (d) 58h
 - (e) D7h
 - (f) 04h
 - (g) 37h
25. False, we do not have to use the accumulator.
26. Yes
27. No, there is not a CJE instruction.
28.
 - (a) CY=1
 - (b) CY=0
 - (c) CY=0
 - (d) CY=0
 - (e) CY=1
 - (f) CY=0
29.
 - (a) Yes
 - (b) Yes
 - (c) No
 - (d) No
 - (e) Yes
 - (f) No

Section 6.4: Rotate Instruction and Data Serialization

30. (a) 01011001b ; 59h
(b) 11100100b; E4h
(c) 00011010b; 1Ah
(d) 10011111b; 9Fh

31.

(a)	RR A
	RR A
	RR A
	RR A
(b)	RL A
	RL A
	RL A
	RL A

32.

MOV R1, #0
MOV R7, #8
MOV A, VALUE
CLR C
AGAIN: RLC A
JC NEXT
INC R1
NEXT: DJNZ R7, AGAIN

33.

MOV R1, #0
MOV R7, #8
MOV A, #68h
CLR C
AGAIN:
RRC A
JC FIRST
INC R1
DJNZ R7, AGAIN
FIRST:
;NOW R1=3 FOR 68h.

34.

MOV R1, #7 ;FOR D7 BIT
MOV R7, #8 ;COUNTER
MOV A, #68h
CLR C
AGAIN:
RLC A
JC FIRST
DEC R1
DJNZ R7, AGAIN
FIRST:
;NOW R1=6 FOR 68H

```

35.      MOV A, #0CCh
      KEEPGOING:
      RR A
      MOV P1, A
      SJMP KEEPGOING

      ;There is no need for the
      ;top nibble, it is just
      ;for convenience.

```

Section 6.5: BCD, ASCII and Other Application Programs

```

36.      ACALL TRANSFER
      ACALL CONVERT
      .....
      .....
TRANSFER:
      MOV     DPTR, MYDATA
      MOV     R1, #60h
      MOV     R3, #4h
NEXT:    CLR     A
      MOVC    A, @A+DPTR
      MOV     @R1,A
      INC     R1
      INC     DPTR
      DJNZ    R3,NEXT
      RET

      ;-----CONVERTING
      ;FROM PACKED BCD TO ASCII
CONVERT:
      MOV     R1,#60H
      MOV     R0,#40H
      MOV     R3,#4
BACK:    MOV     A,@R1
      ANL     A, #0Fh
      ORL     A,#30H
      MOV     @R0,A
      INC     R0
      MOV     A,@R1
      ANL     A, #0F0h
      SWAP    A
      ORL     A,#30H
      MOV     @R0,A
      INC     R0
      INC     R1
      DJNZ    R3, BACK
      RET

      ;-----
      ORG 300h
MYDATA:  DB 76H,87H,98H,43H

```

37.

```

        ACALL TRANSFER
        ACALL CONVERT
        .....
TRANSFER:
        MOV DPTR, MYDATA
        MOV R1, #40h
        MOV R3, #8h
NEXT:   CLR A
        MOVC A, @A+DPTR
        MOV @R1,A
        INC R1
        INC DPTR
        DJNZ R3,NEXT
        RET

;-----CONVERTING
;FROM ASCII TO PACKED BCD
CONVERT:
        MOV R1,#40H
        MOV R0,#60H
        MOV R3,#4
BACK:   MOV A,@R1
        ANL A, #0Fh
        MOV B,A
        INC R1
        MOV A,@R1
        ANL A, #0Fh
        SWAP A
        ORL A,B
        MOV @R0,A
        INC R0
        INC R1
        DJNZ R3, BACK
        RET

;-----
        ORG 300h
MYDATA: DB "87675649"

```

38.

```

        MOV A, #0FFH
        MOV P1, A
        MOV A, P1
        MOV B, #10
        DIV AB
        MOV R0, B
        MOV B, #10
        MOV 40H, R0
        DIV AB
        MOV R0, B
        MOV 41H, R0
        MOV 42H, A
        ORL 40H, #30H
        ORL 41H, #30H
        ORL 42H, #30H
;Since 8D=141, we have 31h,34h,
;and 31h.

```

39. Point 1 if A = 50
 Point 2 if A > 50
 Point 3 if A < 50

40.

```

    ORG    0000h
    MOV    A, #0FFH
    MOV    P1, A
    MOV    A, P1
    ANL    A, #0FH
    MOV    DPTR, TABLE
    MOVC   A, @A+DPTR
    MOV    P2, A
    SJMP   $
    END
TABLE:
    DB    "0123456789ABCDEF"
  
```

41. 'H' = 48h , 'e' = 65h , 'l' = 6Ch , 'o' = 6Fh
 $48h + 65h + 6Ch + 6Ch + 6Fh = 1F4h \rightarrow F4h \rightarrow \text{Check-sum byte} = 0Ch$
42. True (The carry should be discarded)
- 43.

```

    ORG    0000H
    ACALL  LOAD
    ACALL  CALCULATE
    ACALL  TEST
    JNZ    ERROR
    ...
ERROR:
    ...
;-----
LOAD:
    MOV    R0, #30H
    MOV    DPTR, SENTENCE
NEXT1:
    CLR    A
    MOVC   A, @A+DPTR
    MOV    @R0, A
    JZ     EXIT1
    INC    R0
    INC    DPTR
    SJMP   NEXT1
EXIT1:
    RET
;-----
CALCULATE:
    MOV    R0, #30H
    CLR    A
NEXT2:
    ADD    A, @R0
;--CONTINUED ON NEXT PAGE--
  
```

```

        INC     R0
        CJNE    @R0, #0, NEXT2
        CPL     A
        INC     A
        MOV     B, A
        RET

;-----

TEST:
        MOV     A, B
        MOV     R0, #30H
NEXT3:
        ADD     A, @R0
        INC     R0
        CJNE    @R0, #0, NEXT3
        JZ      NO_ERR
        ...ERROR CODE...
NO_ERROR:
        RET

SENTENCE:
        DB "Hello, my fellow"
        DB " World citizens"
        DB 00h

```

44. (a) Modules can easily be used in other programs when needed.
 (b) It is easier to find a bug in the program when it is in a modular format.
 (c) It is much easier to break a heavy task into smaller tasks and get them done one by one in modules.
45. ASCII
- 46.

```

        ORG     0000H
        MOV     A, #0FFH
        MOV     P1, A
        MOV     A, P1
        ANL     A, #0FH
        CLR     C
        SUBB    A, #9
        JNC     CHARACTER
        ADD     A, #9
        ORL     A, #30H
        MOV     P2, A
        SJMP    EXIT

CHARACTER:
        ORL     A, #40H
        MOV     P2, A

EXIT:
        ...

```

CHAPTER 7: 8051 PROGRAMMING IN C

Section 7.1: Data Types and Time Delay in 8051 C

1. a) char b) unsigned char c) unsigned int
 d) unsigned char e) unsigned char f) unsigned char
 g) unsigned int h) int i) unsigned char
2. a) 0E H b) 18 H c) 41 H
 d) 07 H e) 20 H f) 45 H
 g) FF H h) 0F H
3. a) Crystal frequency
 b) Number of clock periods per machine cycle
 c) The way the C compiler compiles (depends on the compiler type)
4. Crystal frequency (also choice over compiler should be taken into consideration)
5. No, that is an internal factor which is not under programmer's control. It is determined and designed by the IC designer.
6. Because in the compilation of the C code into assembly, different compilers use different methods which results in different hex file sizes.

Section 7.2: I/O Programming in 8051 C

7. sbit is used to define a specific bit in a bit-addressable SFR. In addition to SFRs, there are other bit-addressable parts in the RAM memory space and 'bit' should be used (instead of sbit) to access those bit-addressable parts.
8. The following program reverses all bits of P1 in 200msec intervals.

```
#include <reg51.h>
void MSDelay ( unsigned int );
void main ( void ){
    while ( 1 ){
        P1 ^= 0xFF;
        MSDelay ( 200 );
    }
}

void MSDelay ( unsigned int itime ){
    unsigned int i , j;

    for ( i = 0 ; i < itime ; i++ )
        for ( j = 0 ; j < 1275 ; j++ );
}
```


9. To check the shortest time delay which can be produced by the MSDelay() function in the previous exercise, one can call the function using 0 (instead of 200) as the number of milliseconds. By omitting the function totally you can make a shorter delay time in reversing the bits of P1.

10.

```
void delay ( void ){
    unsigned char i;
    unsigned int j;
    for ( i = 0 ; i < 100 ; i++ )
        for ( j = 0 ; j < 1275 ; j++ );
}
```

11.

```
#include <reg51.h>
sbit pin = P1^3;
void delay ( void );

void main ( void ){
    while ( 1 ){
        pin ^= 1;
        delay ();
    }
}

void delay ( void ){
    unsigned char i;
    unsigned int j;
    for ( i = 0 ; i < 200 ; i++ )
        for ( j = 0 ; j < 1275 ; j++ );
}
```

12.

```
#include <reg51.h>
void main ( void ){
    unsigned char j;
    for ( j = 0 ; j < 100 ; j++ )
        P1 = j;
}
```

Section 7.3: Logic Operations in 8051 C

13. a) P1 = 40H b) P1 = 50H c) P1 = 86H
 d) P2 = 90H e) P2 = 60H f) P2 = F0H

- g) P2 = F0H h) P2 = F9H i) P2 = 1EH
j) P2 = 5AH
14. a) P1 = 64H b) P1 = 7BH c) P2 = 3FH
d) P2 = 58H e) P2 = D7H f) P0 = 04H
g) P1 = 37H
15. a) P1 = 19H b) P2 = E4H c) P1 = 1AH
d) P1 = 9CH

16.

```
#include <reg51.h>
unsigned char swap ( unsigned char );
void main ( void ){
    unsigned char myNum = 0x95;
    myNum = swap ( myNum );
    P1 = myNum;
}

unsigned char swap ( unsigned char charNum ){
    unsigned char j;
    j = charNum & 0x0f;
    j = j << 4;
    charNum = charNum >> 4;
    j += charNum;
    return j;
}
```

17. The following program reads the present byte on P2 port continuously and shows the number of zeros on P1.

```
#include <reg51.h>
char zeroCount ( unsigned char );

void main ( void ){
    unsigned char myNum;
    while ( 1 ){
        myNum = P2;
        P1 = zeroCount ( myNum );
    }
}

unsigned char zeroCount ( unsigned char charNum ){
    char j , k = 0;
    for ( j = 0 ; j < 8 ; j++ ){
        if ( ( charNum & 0x01 ) == 0 )
            k++;
        charNum = charNum >> 1;
    }
}
```

18.

```
#include <reg51.h>

void main ( void ){
    unsigned char j = 0x0c;
    while ( 1 ){
        P1 = j;
        j = j >> 1;
        if ( j == 1 )
            j = 0x09;
        if ( j == 4 )
            j = 0x0c;
    }
}
```

Section 7.4: Data Conversion Programs in 8051 C

19. The following program converts the given bytes into two ASCII bytes and sends these two ASCII bytes into ports P1 and P0.

```
#include <reg51.h>

unsigned char bcdNum[4] = { 0x76 , 0x87 , 0x98 , 0x43 };
void BCDtoASCII ( unsigned char );

void main ( void ){
    unsigned char j;
    for ( j = 0 ; j < 4 ; j++ )
        BCDtoASCII ( bcdNum[j] );
    while( 1 );
}

void BCDtoASCII ( unsigned char charNum ){
    unsigned char j;
    j = charNum & 0xf0;
    charNum &= 0x0f;
    charNum += '0';
    P0 = charNum;
    j = j >> 4;
    j += '0';
    P1 = j;
}
```

20. The following program converts the string into BCD format and shows high-byte to P1 and low-byte to P0.

```
#include <reg51.h>

char str[] = "8767";

void main ( void ){
    unsigned char lowByte , highByte;
    lowByte = str[2] - '0';
```

```

        lowByte = lowByte << 4;
        lowByte += str[3] - '0';
        highByte = str[0] - '0';
        highByte = highByte << 4;
        highByte += str[1] - '0';
        P0 = lowByte;
        P1 = highByte;
    }

```

21.

```

#include <reg51.h>

void main ( void ){
    unsigned char j;
    j = P1;
    k = ( j % 10) + '0';
    j /= 10;
    m = ( j % 10 ) + '0';
    j /= 10;
    if ( !j ){           // if ( j == 0 )
        P0 = k;
        P2 = m;
    }
}

```

If $P1 = 10001001b$, it represents decimal number of 137 which cannot be a packed BCD number stored in just one byte. So, in this case the above program will not send any data bytes to P0 and P2.

Section 7.5: Accessing Code ROM Space in 8051 C

22. a) data RAM b) code ROM c) code ROM
 d) code ROM e) data RAM f) data RAM
 g) data RAM h) data RAM i) code ROM
23. The number of data RAM bytes of the original 8051 is limited to 128 bytes. Also in some programs, the microcontroller needs some memory space to be used as the stack, which means the programmer should limit the number of RAM data bytes to something around 100 bytes and leave the rest for the stack.
24. There are basically two main reasons behind that:
- 1) Usually these game characters and shapes occupy a lot of memory space and most probably that space is larger than the RAM memory space.
 - 2) They are usually fixed data bytes that do not change.

25. There is a very limited number of RAM space available in the 8051 microcontroller. So the programmer should always be cautious not to run out of them especially for programs needing a considerable quantity of RAM data bytes for the stack.
26. In some programs a big portion of the code ROM is filled with the constant data bytes, leaving a little space for the instructions (op-codes), in which case the designer might sometimes be forced to use external ROM in the hardware.

27.

```
#include <reg51.h>

code char myName[] = "JOHN SMITH";

void main ( void ){
    unsigned char j = 0;
    while ( myName[j] != '\0' ){
        P2 = myName[j];
        j++;
    }
}
```

CHAPTER 8: 8051 HARDWARE CONNECTION AND INTEL HEX FILE

Section 8.1: Pin Description of the 8051

1. 40
2. Pin 40 for V_{CC} and pin 20 for GND
3. 32 I/O pins
4. 18 and 19
5. 25 MHz
6. Pin number 9
7. input
8. Low, High
9. 0000
10. 07
11. 00
- 12.

	12 MHz	20 MHz	25 MHz	30 MHz
AT89C51	1 μ s	0.6 μ s	0.48 μ s	0.4 μ s
DS5000	0.333 μ s	0.2 μ s	0.16 μ s	0.133 μ s
DS89C4x0	83.3 ns	50 ns	40 ns	33.3 ns

13. External Access, Input
14. V_{CC}
15. output
16. output
17. 8031
18. 8 pins: pin 32-39
19. 8 pins: pin 1-8
20. 8 pins: pin 21-28
21. 8 pins: pin 10-17
22. output
23. P0
24. P1

Section 8.2: Design and Test of DS89C4x0 Trainer

25.

ORG	0000H
MOV	A, #0FFH
MOV	P1, A
MOV	A, P1
MOV	P0, A
MOV	P2, A
MOV	P3, A
SJMP	\$
END	

26. In C language:

```
#include <reg51.h>

void main ( void ){
    data char i;
    P2 = 0xff;
    i = P2;
    P0 = i;
    P1 = i;
}
```

27. Pins 10 and 11, respectively.

28. At memory location 0000, which means that the first byte of code should be placed in 0000h memory location.

29. (a)

```
ORG 0000H
AGAIN:
MOV A, #0FFH
MOV P1, A
MOV P2, A
MOV A, #55H
MOV P1, A
MOV P2, A
SJMP $
END
```

(b)

```
ORG 0000H
MOV A, #55H
AGAIN:
CPL A
MOV P1, A
MOV P2, A
SJMP AGAIN
END
```

30. 0FFFh (4 KBytes)

31. 3FFFh (16 KBytes)

32. 7FFFh (32 KBytes)

33. FFFFh (64 KBytes)

34. Fastest: 33 MHz

35. Slowest: DC

36. $1/33\text{MHz} = 30.3\text{ns}$

37. erase

38. true

39. (a) DS89C4x0 can be programmed through its serial port, eliminating the need for a ROM burner.

(b) Two full-duplex serial ports.

40. It indicates that the loading is good and finished.

Section 8.3: Explaining the Intel Hex File

41. (1) The colon starts the line.
(2) The first byte after the colon (10h) indicates that there will be 16 data bytes in this line.
(3) The next two bytes (0000h) indicate the ROM address that the first data byte should be burned into.
(4) The next byte (00h) shows that this is not the last line of the program.
(5) The next sixteen bytes are the op-codes and their operand data.
(6) And the last byte is the check-sum byte of the previous bytes in the line.
42. The only difference in this line from the previous line is in the third part (0010). 0010 indicates that the first data byte in this line must be stored in the ROM location 0010h.
43. Calculation of the check-sum byte:
 $10h + 00h + 00h + 00h + 75h + 80h + 55h + 75h + 90h + 55h + 75h + A0h + 55h + 7Dh + FAh + 11h + 1Ch + 75h + 80h + AAh = 761h$
Dropping the carries: 61h
2's complement of 61h: 9Fh
Verification of the check-sum byte: $10h + 00h + \dots + 80h + AAh + 9Fh = 800h$
Dropping the carries: 00h, so 9Fh is the correct check-sum byte of the line.
44. $10h + 00h + 10h + 00h + 75h + 90h + AAh + 75h + A0h + AAh + 7Dh + FAh + 11h + 1Ch + 80h + E4h + 7Ch + 23h + 7Bh + 4Fh = 7FFh$
Dropping the carries: FFh
2's complement of FFh: 01h
Verification of the check-sum byte: $10h + \dots + 4Fh + 01h = 800h$
Dropping the carries: 00h
- 45.

:1001000075805575905575A0557DFA311C7580AA7E
:100110007590AA75A0AA7DFA311C80E47C237B4FE0
:07012000DBFEDCFADDF62234
:000000001FF

Differences in the first line:

- (a) The AAAA part has changed from 0000 to 0100, which is the new starting address.
- (b) The new address of the Delay subroutine is 011C, and that is why the op-code has changed from 11 to 31.
- (c) The above changes ((a) and (b)) in the hexadecimal numbers, have changed the check-sum byte to 7Eh.

The same changes can be seen in the second line of the .HEX file.

In the third line only the AAAA part has changed from 0020 to 0120, and the SS byte has changed accordingly.

The fourth line has remained unchanged.

46.

```
:1003000075805575905575A0557DFA711C7580AA3C
:100310007590AA75A0AA7DFA711C80E47C237B4F9E
:07032000DBFEDCFADDF62232
:00000001FF
```

Differences in the first line:

- (a) The AAAA part has changed from 0000 to 0300, which is the new starting address.
- (b) The new address of the Delay subroutine is 031C, and that is why the op-code has changed from 11 to 71.
- (c) The above changes ((a) and (b)) in the hexadecimal numbers, have changed the check-sum byte to 3Ch.

The same changes can be seen in the second line of the .HEX file.

In the third line only the AAAA part has changed from 0020 to 0320, and the SS byte has changed accordingly.

The fourth line has remained unchanged.

47.

```
ORG    0000H
MAIN:
MOV     P1, #55h
MOV     P2, #55h
MOV     P1, #0AAh
MOV     P2, #0AAh
SJMP    MAIN
END
```

The Intel .HEX file would be as follows:

```
:0E00000075905575A0557590AA75A0AA80F24E
:00000001FF
```

The program consists of 14 bytes of op-codes and operands. That is why the first byte after ':' is 0Eh

The first address of the ROM to be filled is 0000h.

It is not the last line of the .HEX file, so a 00 byte is put afterwards.

14 bytes of op-codes and operands follow.

The last byte is the 2's complement of B2h, since the sum of all other bytes of the line is 7B2h.

CHAPTER 9: 8051 TIMER PROGRAMMING IN ASSEMBLY AND C

Section 9.1: Programming 8051 Timers

1. 2
2. 16, timer 0, timer 1
3. TL0 and TH0
4. TL1 and TH1
5. No
6. 8
7. TMOD is used to set the timer modes. It is also used to indicate the clock source.
8. False, it is not bit addressable.
9. 00100010b
10. (a) $f = 921.6 \text{ KHz}$ $T = 1.085 \mu\text{s}$ (c) $f = 2.000 \text{ MHz}$ $T = 0.500 \mu\text{s}$
(b) $f = 1.666 \text{ MHz}$ $T = 0.600 \mu\text{s}$ (d) $f = 2.500 \text{ MHz}$ $T = 0.400 \mu\text{s}$
11. (a) 13 bit
(b) 16 bit
(c) 8 bit
12. (a) 1FFFh (8191)
(b) FFFFh (65535)
(c) FFh (255)
13. (a) when it rolls over to 0000h from 1FFFh
(b) when it rolls over to 0000h from FFFFh
(c) when it rolls over to 00h from FFh
14. True, they both have their own TF.
15. True, they both have their own TR.
16. $\text{FFFF} - 1\text{C12} = \text{E3EDH} = 58349$ and $58349 \times 1.085 \mu\text{s} = 63.308 \text{ ms}$
17. $\text{FFFF} - 1\text{C12} = \text{E3EDH} = 58349$ and $58349 \times 0.75 \mu\text{s} = 43.761 \text{ ms}$
18. $\text{FFFF} - \text{F210} = \text{DEFH} = 3567$ and $3567 \times 1.085 \mu\text{s} = 3.87 \text{ ms}$
19. $\text{FFFF} - 1\text{C12} = \text{E3EDH} = 58349$ and $58349 \times 0.60 \mu\text{s} = 35.010 \text{ ms}$
20. F8CDh since $2000 \text{ usec} / 1.085 \mu\text{s} = 1843$ and $(-1843) = \text{F8CDh}$
21. E5F6h since $5000 \text{ usec} / 0.75 \mu\text{s} = 6666$ and $(-6666) = \text{E5F6h}$

22.

MOV TL0, #00h MOV TH0, #0F7h

23.

MOV TL1, #48h MOV TH1, #0FFh

24.

MOV R3, #0AH MOV TL1, #0E6H MOV TH1, #0BEH SETB TR1 HERE: JNB TF1, HERE CLR TF1 DJNZ R3, HERE CLR TR1
--

25. 7 Hz since $1/[2 \times 65536 \times 1.085 \mu\text{s}] = 7 \text{ Hz}$
26. 460 KHz since $1/[2 \times 1.085 \mu\text{s}] = 460 \text{ KHz}$
27. 10 Hz since $1/[2 \times 65536 \times 0.75 \mu\text{s}] = 10 \text{ Hz}$
28. 666.667 KHz since $1/[2 \times 0.75 \mu\text{s}] = 666.667 \text{ KHz}$
29. F1h through FFh, and then 00h, 15 steps
- 30.

```
;Since 500  $\mu\text{s}$  /1.085  $\mu\text{s}$  = 460 and (-460) = FE34
;we have the following:
```

```
        MOV TMOD, #10H
HERE:   MOV TL1, #34H
        MOV TH1, #0FEH
        SET TR1
BACK:   JNB TF1, BACK
        CLR TR1
        CPL P1.0
        CLR TF1
        SJMP HERE
```

31.

```
        MOV TMOD, #01
HERE:   MOV TL0, #67H
        MOV TH0, #0FFH
        SET TR0
BACK:   JNB TF0, BACK
        CLR TR0
        CPL P1.0
        CLR TF0
        SJMP HERE
```

32.

```
        MOV TMOD, #01
HERE:   MOV TL0, #7DH
        MOV TH0, #0F9H
        SET TR0
BACK:   JNB TF0, BACK
        CLR TR0
        CPL P1.0
        CLR TF0
        SJMP HERE
```

33.

```
        MOV TMOD, #10
HERE:   MOV TL1, #0ADH
        MOV TH1, #0FFH
        SET TR1
BACK:   JNB TF1, BACK
        CLR TR1
        CPL P1.1
        CLR TF1
        SJMP HERE
```

34.

```

MOV    TMOD, #10H

MOV    R1, #20
HERE:  MOV    TL1, #0FDH
      MOV    TH1, #04BH
      SETB   TR1
BACK:  JNB    TF1, BACK
      CLR    TR1
      CLR    TF1
      DJNZ   R1, HERE

```

35.

```

MOV    TMOD, #10H

MOV    R1, #10
HERE:  MOV    TL1, #0CBH
      MOV    TH1, #7DH
      SETB   TR1
BACK:  JNB    TF1, BACK
      CLR    TR1
      CLR    TF1
      DJNZ   R1, HERE

```

36. 1.8 KHz since $1/[2 \times 256 \times 1.085 \mu\text{s}] = 1.8 \text{ KHz}$.

37. 461 KHz since $1/[2 \times 1.085 \mu\text{s}] = 460 \text{ KHz}$.

38. 2.6 KHz since $1/[2 \times 256 \times 0.75 \mu\text{s}] = 2.6 \text{ KHz}$.

39. 666 KHz since $1/[2 \times 0.75 \mu\text{s}] = 666 \text{ KHz}$.

40. (a) F4h (e) 88h
 (b) EAh (f) 98h
 (c) DEh (g) 22h
 (d) A4h (h) BDh
41. (a) 12 (e) 120
 (b) 22 (f) 104
 (c) 34 (g) 222
 (d) 92 (h) 67

42. (a) $12 \times 1.085 \mu\text{s} = 13 \mu\text{s}$
 (b) $24 \mu\text{s}$ since $22 \times 1.085 \mu\text{s} = 24 \mu\text{s}$
 (c) $36.89 \mu\text{s}$ (f) $112.84 \mu\text{s}$
 (d) $99.82 \mu\text{s}$ (g) $240.87 \mu\text{s}$
 (e) $130 \mu\text{s}$ (h) $72.69 \mu\text{s}$

Section 9.2: Counter Programming

43. high
 44. Yes
 45. P3.4 (pin 14 in DIP package)
 46. P3.5 (pin 15 in DIP package)

47.

```

        MOV    TMOD, #50h
        SETB   P3.5
AGAIN:  MOV    TL1, #0E0H
        MOV    TH1, #0B1H
        SETB   TR1
BACK:   MOV    A, TL1
        MOV    P1, A
        MOV    A, TH1
        MOV    P1, A
        JNB    TF1, BACK
        CLR    TR1
        CLR    TF1
        SJMP   AGAIN

```

48.

```

        MOV    TMOD, #06h
        MOV    TL0, #-20
        MOV    TH0, #-20
        SETB   P3.4
AGAIN:  SETB   TR0
BACK:   MOV    A, TL0
        MOV    P2, A
        JNB    TF0, BACK
        CLR    TR0
        CLR    TF0
        SJMP   AGAIN

```

49.

```

        MOV    TMOD, #50h
        MOV    TL1, #-99
        MOV    TH1, #-99
        SETB   P3.5
AGAIN:  SETB   TR1
BACK:   MOV    A, TL1
        ACALL  CONVERT
        JNB    TF1, BACK
        CLR    TR1
        CLR    TF1
        SJMP   AGAIN
;--This will convert from binary (hex)
;to decimal and send each digit to the port
CONVERT:
        MOV    B, #10
        DIV    AB
        MOV    P0, B
        MOV    B, #10
        DIV    AB
        MOV    P1, B
        MOV    P2, A
        RET

```

50. 8-bit

51. False

52. SETB TCON.4

Section 9.3: Programming Timers 0 and 1 in 8051 C

53. $T = 333.3 \mu s \rightarrow T/2 = 166.7 \mu s \rightarrow 166.7 \mu s / 1.085 \mu s = 153 \rightarrow TH0, TL0 = 67h$ in auto-reload mode.

```
#include <reg51.h>

sbit mybit = P1^5;

void main ( void ){
    TMOD = 0x02;
    TH0 = 0x67;
    TL0 = 0x67;
    TR0 = 1;
    while(1){
        while(!TF0);
        TF0 = 0;
        mybit = ~mybit;
    }
}
```

54. Exactly the same as the previous problem, except for TMOD which must be assigned the value of 0x20, instead of 0x02. Obviously, TH0, TL0, TR0, and TF0 should be changed with TH1, TL1, TR1, and TF1, respectively.
55. $T = 2ms, T/2 = 1ms, 1ms / 1.085 \mu s = 921 = 399 \rightarrow TH0 = FCh, TL0 = 67h$

```
#include <reg51.h>

sbit mybit = P1^5;
void delay ( void );

void main ( void ){
    while ( 1 ) {
        mybit = ~mybit;
        delay();
    }
}

void delay ( void ){
    TMOD = 0x01;
    TL0 = 0x67;
    TH0 = 0xfc;
    TR0 = 1;
    while ( !TF0 );
    TR0 = 0;
    TF0 = 0;
}
```

56. The same as the previous problem, except for TMOD, which must be 0x10 instead of 0x01, and also TL0, TH0, TR0, and TF0 should be replaced with TL1, TH1, TR1, and TF1.

57.

```
#include <reg51.h>

void main ( void ){
    TMOD = 0x50;
    TL1 = 0x20;
    TH1 = 0x4e;
    TR1 = 1;
    while ( 1 ) {
        P1 = TL1;
        P2 = TH1;
        if ( TF1 ){
            TF1 = 0;
            TL1 = 0x20;
            TH1 = 0x4e;
        }
    }
}
```

58.

```
#include < reg51.h>
void main ( void )
{
    TMOD = 0x06;
    TL0 = 20;
    TH0 = 20;
    while(1)
    {
        P2 = TL0;
        if(TF0)
            TF0 = 0;
    }
}
```

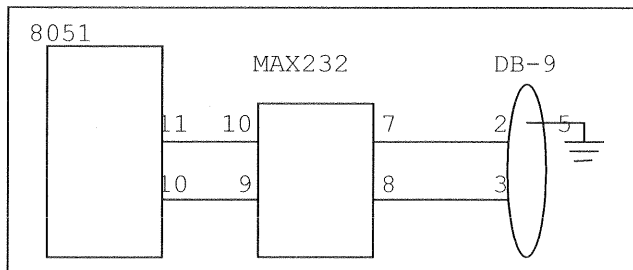
CHAPTER 10: 8051 SERIAL PORT PROGRAMMING IN ASSEMBLY AND C

Section 10.1: Basics of Serial Communication

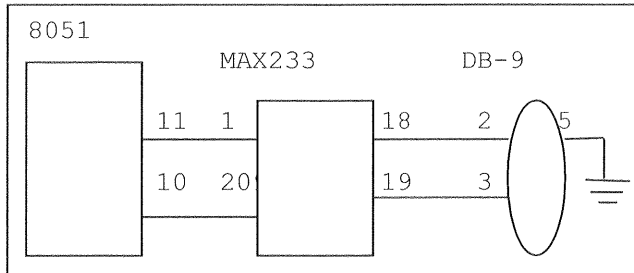
1. Parallel
2. False
3. 1010110100 ---> this start bit goes first
4. Mark
5. False. It can be 1 or 2 but we always need stop bit.
6. 7 bit data + 1 stop bit + 1 start bit = 9 bits
 $2/9 \times 100 = 22\%$
7. False
8. It makes the RS232 standard compatible with TTL logic level.
9. False
10. 9 pins, since the rest are not involved in data transfer.
11. False
12. 3: TxD RxD Gnd
13. DTE -to- DTE
14. DCD, RxD, TxD, DTR, Gnd, DSR, RTS, CTS, RI
15. 4,000,000 bits (200 x 80 x 25 x 10)
16. 416.6 seconds

Section 10.2: 8051 Connection to RS232

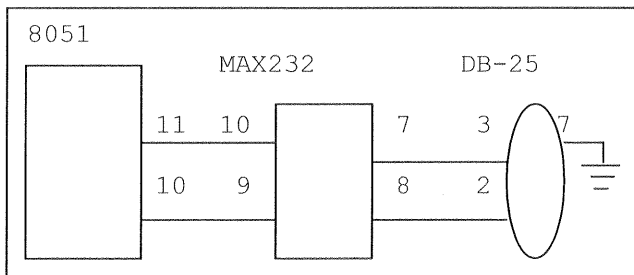
17. 16
18. VCC = 16, GND = 15
19. 20
20. VCC = 7, GND = 6
21. No
22. The MAX232 does not require an external capacitor, but it is more expensive.
23. 2
24. 2
- 25.



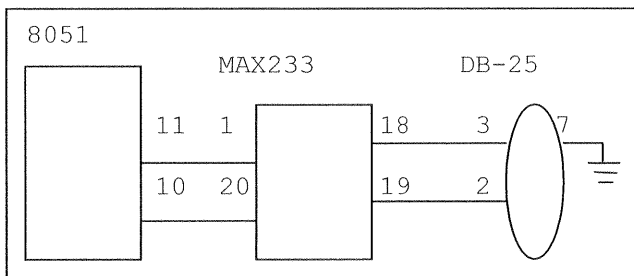
26.



27.



28.



Section 10.3: 8051 Serial Port Programming in Assembly

29. (a), (c), (e), (f)
30. Timer 1
31. Mode 2
32. To send a byte serially it must be placed in the SBUF register.
33. 8
34. It controls the framing of the data for serial ports.
35. 8
36. First we have $11.0592 \text{ MHz} / 12 = 921.6 \text{ KHz}$ and $921.6 / 32 = 28,800$.
Now each is calculated by dividing 28,800 by the baud rate.
E.g. $28800 / 9600 = 3$ and $(-3) = \text{FDh}$
(a) $-3 = \text{FDh} = 253$
(b) $28800 / 4800 = 6$ and $(-6) = \text{FAh} = 250$
(c) $28800 / 1200 = 24$ and $(-24) = \text{E8h} = 232$
(d) $\text{A0h} = 160$
(e) $40\text{h} = 64$

37. 28800

```
38.      MOV TMOD, #20H
        MOV TH1, #-24
        MOV SCON, #50H
        SETB TR1
AGAIN:  MOV A, #'Z'
        MOV SBUF, A
HERE:   JNB TI, HERE
        CLR TI
        SJMP AGAIN
```

39.

```
      MOV DPTR, MYDATA
      MOV R2, #52          ;COUNT
      MOV A, PCON
      SETB ACC.7           ;MAKE IT 56k RATE
      MOV PCON, A
      MOV TMOD, #20H
      MOV TH1, #-1
      MOV SCON, #50H
      SETB TR1
A_2:   CLR A
      MOVC A, @A+DTPR
      MOV SBUF, A
H_1:   JNB TI, H_1
      INC DPTR
      CLR TI
      DJNZ R2, A_2

      ORG 300H
MYDATA: DB "The earth is... citizen"
```

- 40. at the beginning of the stop bit of the outgoing framed byte
- 41. midway through the stop bit of the incoming framed byte
- 42. SCON, Yes
- 43. REN enables or disables serial data reception
- 44. CLR SCON.4
- 45. It belongs to the PCON register. If SMOD = 0, the maximum baud rate is 28,800 (XTAL = 11.0592 MHz). If SMOD = 1, the maximum baud rate is 56K. SMOD allows us to double the serial data transfer rate with the same crystal.
- 46. Low
- 47. $6\text{MHz}/12 = 1.333\text{ MHz}$. $1.333\text{MHz}/32 = 41666\text{ Hz}$
 - (a) $41666/10 = 4166$
 - (b) $41666/25 = 1666$
 - (c) $41666/200 = 208$
 - (d) $41666/180 = 231$

48. $24\text{MHz}/12 = 2\text{MHz}$. $2\text{MHz}/32 = 62500\text{ Hz}$
 (a) $62500/15 = 4166$
 (b) $62500/24 = 2604$
 (c) $62500/100 = 625$
 (d) $62500/150 = 416$
49. $16\text{MHz}/12 = 1.333\text{ MHz}$ $1.333\text{MHz}/16 = 83333\text{ Hz}$
 (a) $83333/10 = 8333$
 (b) $83333/25 = 3333$
 (c) $83333/200 = 416$
 (d) $83333/180 = 462$
50. $24\text{MHz}/12 = 2\text{ MHz}$ $2\text{ MHz}/16 = 125000\text{ Hz}$
 (a) $125000/15 = 8332$
 (b) $125000/24 = 5208$
 (c) $125000/100 = 1250$
 (d) $125000/150 = 832$

Section 10.4: Programming the Second Serial Port

51. Timer 1
 52. Timer 1
 53. It should be set into auto-reload mode of operation.
 54. It has the same role as SBUF in the early 8051, but SBUF1 represents the serial buffer for the second serial port.
 55. 8-bit register.
 56. It has the same role as SCON in 8051, but SCON1 is used for controlling the operation of the second serial port.
 57. 8-bit register.
 58. Input Frequency = $11.0592\text{ MHz} / 32 = 345600\text{ Hz}$.
 (a) $345600 / 9600 = 36 = 24\text{H}$
 $10000\text{H} - 24\text{H} = \text{FFDCH}$, TH1 = FFH, TL1 = DCH
 (b) $345600 / 4800 = 72 = 48\text{H}$
 $10000\text{H} - 48\text{H} = \text{FFB8H}$, TH1 = FFH, TL1 = B8H
 (c) $345600 / 2400 = 144 = 90\text{H}$
 $10000\text{H} - 90\text{H} = \text{FF70H}$, TH1 = FFH, TL1 = 70H
 (d) $345600 / 300 = 1152 = 480\text{H}$
 $10000\text{H} - 480\text{H} = \text{FB80H}$, TH1 = FBH, TL1 = 80H
 (e) $345600 / 150 = 2304 = 900\text{H}$
 $10000\text{H} - 900\text{H} = \text{F700H}$, TH1 = F7H, TL1 = 00H

59.

MOV	TMOD, #20H
MOV	TH1, #-24
MOV	SCON1, #50H
SETB	TR1
AGAIN:	
MOV	SBUF1, #'Z'
LOOP: JNB	TI1, LOOP
CLR	TI1
SJMP	AGAIN

60.

```

        MOV    DPTR, MYDATA
        MOV    R2, #52                ;COUNT
        MOV    T2CON, #30H
        MOV    RCAP2L, #0FAH
        MOV    RCAP2H, #0FFH
        MOV    TL2, #0FAH
        MOV    TH2, #0FFH
        MOV    SCON, #50H
        SETB   TR2
A_2:    CLR    A
        MOVC   A, @A+DTPR
        MOV    SBUF, A
H_1:    JNB    TI, H_1
        INC    DPTR
        CLR    TI
        DJNZ   R2, A_2
        MOV    DPTR, MYDATA
        MOV    R2, #52
        SJMP   A_2

        ORG 300H
MYDATA: DB "The earth is... citizens"

```

61. When the second serial port of a microcontroller finishes sending a byte of data, TI1 flag gets raised.

Section 10.5: Serial Port Programming in C

62.

```

#include <reg51.h>
void main ( void ){
    TMOD = 0x20;
    TH1 = 0xE8;
    SCON = 0x50;
    TR1 = 1;
    while (1){
        SBUF = 'Z';
        while ( !TI );
        TI = 0;
    }
}

```

63.

```
#include <reg51.h>
code char str[] = "The earth ... citizens";
void main ( void ){
    unsigned char j = 0;

    TMOD = 0x20;
    TH1 = 0xFF;

    SCON = 0x50;
    PCON = PCON | 0x80;

    TR1 = 1;
    while (1){
        while ( str[j] != '\0' ){
            SBUF = str[j];
            while ( !TI );
            TI = 0;
            j++;
        }
        j = 0;
    }
}
```

64.

```
#include <reg52.h>
sfr SBUF1 = 0xC1;
sfr SCON1 = 0xC0;
sbit TI1 = 0xC1;
void main(void)
{
    TMOD = 0x20;
    TH1 = 0xE8;
    SCON1 = 0x50;
    TR1 = 1;
    while (1)
    {
        SBUF1 = 'Z';
        while(!TI1);
        TI1 = 0;
    }
}
```

65.

```
#include <reg51.h>
code char str[] = "The earth ... citizens";
sfr T2CON = 0xC8;
sfr TL2 = 0xCC;
sfr TH2 = 0xCD;
sfr RCAP2L = 0xCA;
sfr RCAP2H = 0xCB;
sbit TR2 = 0xCA;

void main ( void ){
    unsigned char j = 0;

    T2CON = 0x30;
    SCON = 0x50;
    TH2 = 0xFF;
    TL2 = 0xFA;
    RCAP2H = 0xFF;
    RCAP2L = 0xFA;
    TR2 = 1;
    while (1){
        while ( str[j] != '\0' ){
            SBUF = str[j];
            while ( !TI );
            TI = 0;
            j++;
        }
        j = 0;
    }
}
```

CHAPTER 11: INTERRUPTS PROGRAMMING IN ASSEMBLY AND C

Section 11.1: 8051 Interrupts

1. Interrupt
2. 6
3. 0000 to 0025H
4. True
5. 0003h
6. 0013h
7. 000Bh
8. 001Bh
9. 0023h
10. This is done to skip the memory space assigned to the interrupt vector table.
11. All 0's. All interrupts are masked which means that no interrupt will be responded to.
12. MOV IE, #10001100b
13. MOV IE, #9Fh
14. Pins 12, 13
15. 8 bytes for each
16. 8 bytes for each
17. 8 bytes
18. False, it is bit addressable
19. CLR IE.7
20. CLR IE.2
21. False
22. 3 bytes, so that we can place an LJMP (3-byte instruction) in order to jump to the start of our code located anywhere in the 64KB ROM space.

Section 11.2: Programming Timer Interrupts

23. True
24. 001Bh
25. D1, "SETB IE.1"
26. D3, "SETB IE.3"
27. It counts up from F0h to FFh and when the rollover from FF to 00 occurs, the interrupt is activated. Then the microcontroller jumps to ROM address 000B to execute the interrupt service routine.
28. True
29. It counts up from FFF8h to FFFFh and when the rollover from FFFF to 0000 occurs, the interrupt is activated. Then the microcontroller jumps to ROM address 001B to execute the interrupt service routine.
30. It counts up to FFh and when the rollover from FF to 00 occurs, the interrupt is activated. Then the microcontroller jumps to ROM address 001B to execute the interrupt service routine.

31.

```
;XTAL=11.0592 MHz is assumed.
    ORG 0
    LJMP MAIN
    ORG 0BH
    CPL P2.2
    RETI

;-----
    ORG 30H
MAIN:
    MOV    P1,0FFH ;P1 BITS AS INPUT
HERE: MOV R4,#14
B2:  MOV R5,#255
B1:  MOV A,P1
     MOV P0,A
     DJNZ R5,B1
     DJNZ R4,B2
     MOV IE, #82H ;after 160 ms enable timer 0 interrupt
     SETB TF0    ;force the interrupt by software
     SJMP HERE
```

32.

```
; XTAL=11.0592 MHz. is assumed

    ORG 0
    LJMP MAIN
    ORG 0BH
    LJMP ISR_TOGL

;-----
    ORG 30H
MAIN:
    MOV    P1,0FFH      ;P1 BITS AS INPUT
;----- 2-SEC DELAY
HERE: MOV R3,#12
B3:  MOV R4,#150
B2:  MOV R5,#255
B1:  MOV A,P1
     MOV P0,A
     DJNZ R5,B1
     DJNZ R4,B2
     DJNZ R3,B3
     MOV IE, #82H ;after 2-sec enable timer 0 interrupt
     SETB TF0    ;force the interrupt by software
     SJMP HERE

;-----
ISR_TOGL:
    MOV R2,#4
BACK: CLR    P2.7
     ACALL DELAY      ;50 ms DELAY
     SETB P2.7
     ACALL DELAY
     DJNZ R2,BACK
     RETI
```


Section 11.3: Programming External Hardware Interrupts

- 33. False
- 34. 03H, P3.2 (PIN 12); 13H, P3.3 (PIN 13)
- 35. IE.0 , SETB IE.0
- 36. IE.2,SETB IE.2
- 37. MOV IE, #10000101B
- 38. An external pulse triggers the interrupt and the microcontroller jumps to ROM address 0003 to execute the interrupt service routine. We must deactivate the interrupting pulse before the RETI instruction is executed in order to prevent multiple interrupts from the same pulse.
- 39. False. It is level trigger.
- 40. by changing from level triggered to edge triggered
- 41. TCON.0 , TCON.1
- 42. TCON.2 and TCON.3
- 43. False. It does not need to clear the TCON.3 since this is done by the CPU automatically when RETI is executed.
- 44. One sets the edge or level triggered option, while the other one is set when an H-to-L pulse is detected on the EX pin.
- 45. One sets the edge or level triggered option, while the other one is set when an H-to-L pulse is detected on the EX pin
- 46. An external pulse triggers the interrupt and the microcontroller jumps to ROM address 0013h to execute the interrupt service routine. We must deactivate the interrupting pulse before the RETI instruction is executed in order to prevent multiple interrupts from the same pulse.
- 47.

```
; XTAL=11.0592 MHz is assumed.
ORG 0
LJMP MAIN
ORG 0BH
CPL P2.2
RETI
ORG 30H
MAIN:
MOV TMOD, #02H
MOV TH0, -153
MOV IE, #82H
SETB TR0
MOV P1, #0FFH      ;P1 as input
HERE: MOV A, P1
MOV P2, A
SJMP HERE
```

48.

```

;We feed a 1Hz square wave to P3.3

    ORG 0
    LJMP MAIN
    ORG 013H
    CPL P0.4
    RETI

;-----
    ORG 30H
MAIN:
    SETB P3.3 ;P3.3 as input to receive 1-sec pulses
    SETB TCON.2 ;edge trigger
    MOV IE, #84H
    MOV P1, #0FFH ;P1 as input
HERE: MOV A, P1
      MOV P2, A
      SJMP HERE

```

49. The edge trigger occurs on the negative edge of the pulse applied to the external pins of P3.2 and P3.3. No new interrupt is recognized until the RETI instruction is executed. The level triggered interrupt occurs when a low level pulse is applied to these pins. Within 4 clock cycles, the pulse must become high, or before the execution of the RETI in order to prevent multiple interrupts from the same pulse.
50. By activating the bits in the TCON register using instructions such as “SETB TCON.0” and “SETB TCON.2” for EX0 and EX1, respectively.
51. both hardware interrupts INT0 and INT1
52. TCON

Section 11.4: Programming the Serial Communication Interrupt

53. False
54. 0023h; at least 3 bytes are needed but we can use more depending where the MAIN subroutine starts.
55. D4 bit, “MOV IE,#10010000b”
56. The RI flag is raised when the entire frame of data including the stop bit is received. As a result, the received byte is delivered to the SBUF register and the 8051 jumps to memory location 0023h to execute the ISR belonging to this interrupt. In the serial COM ISR, we must save the SBUF content before it is lost (overwritten) by the incoming data.
57. True (for a transmit interrupt)
58. True
59. False. The instruction should be “CLR RI” instead of “CLR TI.”
60. When the last bit of the framed data is transferred, the TI is raised by the microcontroller indicating that the SBUF register is ready for the next byte. As a result of TI being raised, the microcontroller jumps to ROM address 23h to execute the interrupt service routine.

61. When the entire frame of data is received and given to the SBUF register, RI is raised indicating that the SBUF register has a byte of data that must be picked up before it is overrun. As a result of RI being raised, the microcontroller jumps to ROM address 23h to execute the interrupt service routine.

62.

```
ORG 0
LJMP MAIN
ORG 000BH
CPL P0.1
RETI
ORG 23H
LJMP SERIAL

ORG 30H
MAIN: MOV TMOD, #22H
      MOV TH1, #0FDH
      MOV SCON, #50H
      MOV TH0, #-92
      MOV IE, #92H
      SET TR1
      SET TR0
BACK: SJMP BACK

ORG 100H
SERIAL: JB TI, TRANS
        MOV A, SBUF
        MOV P2, A
        CLR RI
        RETI
TRANS: CLR TI
        RETI
        END
```

63.

```

    ORG 0
    LJMP MAIN
    ORG 23H
    LJMP SERIAL
    ORG 30H
MAIN:
    MOV TMOD, #20H
    MOV TH1, #-3          ;9600 baud rate
    MOV SCON, #50H
    MOV IE, #90H
    SET TR1
HERE: MOV R3, #6          ;1-SEC
B3:   MOV R4, #150
B2:   MOV R5, #255
B1:   NOP
      NOP
      DJNZ R5, B1
      DJNZ R4, B2
      DJNZ R3, B3
      CPL P1.6
      SJMP HERE
;-----
    ORG 200H
SERIAL: JB TI, TRANS
        MOV A, SBUF
        MOV P2, A
        CLR RI
        RETI
TRANS:  CLR TI
        RETI
        END

```

Section 11.5: Interrupt Priority in the 8051/52

64. True. External interrupt 0 has the highest priority.
65. IP register. It allows the programmer to change the microcontroller priorities. By setting high the designated interrupt bit, we can assign the highest priority to it.
66. SETB IP.2 will assign the highest priority to external interrupt 1.
67. SETB IP.3
68. SETB IP.0 will assign the highest priority to external interrupt 0
69. INT0 is serviced first.
70. TF0 is serviced first.
71. TF0 is serviced first.
72. INT0 is serviced first.
73. The microcontroller finishes the higher priority first and then the lower priority interrupt is served.

CHAPTER 12: LCD AND KEYBOARD INTERFACING

Section 12.1: LCD Interfacing

1. 8
2. They are the control lines for the LCD. E is for latching information into the LCD, R/W is for reading from or writing information into the LCD, and RS is for making a distinction between the data and command.
3. Vee controls the contrast of the screen while Vcc provides power to the LCD.
4. command, 01h
5. 0Fh
6. RS = 0, RW = 0, E = H-to-L pulse
7. RS = 1, RW = 0, E = H-to-L pulse
8. (a)
9. True
10. Sending information to the LCD without checking the busy flag is very simple to write but the microcontroller wastes a lot of time in a delay subroutine. The advantage of monitoring the busy flag is that the next information is sent when the LCD is ready and there is no wasting of time. This is very critical, especially when the microcontroller is busy and must serve many devices. However, the subroutine for monitoring the busy flag is much harder to write and takes more code. To monitor the busy flag, we make the port D0 - D7 of the LCD an input port, read the D7 bit, and wait for low.
11. 80h is the starting location, and 16 locations to the right is 8Fh.
12. From Table 12-3 we have C0h, which is first location of line 2.
13. 80h is the starting location, and 20 locations to the right is 13h which result in $80h + 13h = 93h$.
14. This is 2 locations from C0h since C0 is the first location of line 2.
15. If 80h is the address of the first location, then adding 39 (27h) we get $80h + 27h = A7h$.
16. If C0h is the address of the first location, then adding 39 (27h) we get $C0h + 27h = E7h$.
17. 89h since $80h + 9 = 89h$. The 10th is address 9 since it starts at 0.
18. $C0h + 13h = D3h$. Notice that the 20th starts at 0 and goes to 13h.
- 19.

```
COMNWRT:
    MOV P2, A
    CLR P1.4
    CLR P1.5
    SETB P1.6
    CLR P1.6
    RET
```

20.

```

        MOV DPTR, #MYDATA
HERE:
        CLR A
        MOVC A, @A+DPTR
        JZ     DONE
        ACALL DISPLAY_DATA
        INC DPTR
        SJMP HERE
DONE: SJMP DONE

DISPLAY_DATA:
        ACALL READY
        MOV P2, A
        SETB P1.4    ;RS=1 FOR DATA
        CLR P1.5     ;R/W=0 FOR WRITE
        SETB P1.6     ;H-TO-L FOR E
        CLR P1.6
        RET

READY:
        SETB P2.7    ;P2.7=INPUT TO READ BUSY FLAG
        CLR P1.4     ;RS=0
        SETB P1.5     ;R/W=1 FOR READ
BACK: CLR P1.6
        SETB P1.6
        CLR P1.6
        JB P2.7, BACK
        RET
;-----
        ORG 300H
MYDATA: DB "Hello",0

```

Section 12.2: Keyboard Interfacing

21. 1s
22. (a)
23. (b)
24. (a) column D3, the row is unknown
(b) column D0, the row is unknown
25. (a) Ground all the rows at once.
(b) Read the columns.
(c) Check to see if any of the columns is low.
26. (a) Ground one row at the time.

- (b) Read the columns and check to see if any of the bits is low. If it is, load the address of the look-up table for that row.
 - (c) Rotate the column bits through the carry and check for low to identify the key.
27. This allows the microcontroller to do other things.
 28. Use a chip for the keyboard (for both key press and identification) and then pass the scan code for the pressed key to the microcontroller via a hardware interrupt.

CHAPTER 13: ADC, DAC, AND SENSOR INTERFACING

Section 13.1: Parallel and Serial ADC

1. CS = 0, WR = L-to-H
2. CS = 0, WR = H-to-L
3. It keeps the information internally and informs us by making the INTR pin go high when the conversion is finished.
4. the information is over written
5. Output. It indicates that analog data has been converted and it is ready to be sent out (end of conversion).
6. (a) 10 mV, since $1.28 \text{ V} \times 2 = 2.56 \text{ V}$ and $2.56 \text{ V} / 256 = 10 \text{ mV}$.
(c) (b). 7.81 mV, since $1 \text{ V} \times 2 = 2 \text{ V}$ and $2 \text{ V} / 256 = 7.8 \text{ mV}$.
(d) (c) 14.8 mV, since $1.9 \text{ V} \times 2 = 3.8 \text{ V}$ and $3.8 \text{ V} / 256 = 14.8 \text{ mV}$.
7. 2.56V, since $20 \text{ mV} \times 256 = 5.12 \text{ V}$ and one half of it is 2.56V.
8. 0.64 V, since $5 \text{ mV} \times 256 = 1.28 \text{ V}$ and one half of it is 0.64V.
9. Vin(+) is the positive side of the analog input voltage and its negative side is Vin(-).
10. 5 V ($255 \times 19.53 \text{ mV} = 5 \text{ V}$)
11. (a) since the step size is 5 mV and all inputs are high, we get $255 \times 5 \text{ mV} = 1.275 \text{ V}$.
(b) $10011001 = 153$ and $153 \times 5 \text{ mV} = 0.756 \text{ V}$.
(c) $1101100 = 108$ and $108 \times 5 \text{ mV} = 0.540 \text{ V}$.
12. True
13. (d)
14. (d)
15. (a) $(5 \text{ V}) / (2^8) = 19.53 \text{ mV}$
(a) $(5 \text{ V}) / (2^{10}) = 4.88 \text{ mV}$
(a) $(5 \text{ V}) / (2^{12}) = 1.22 \text{ mV}$
(a) $(5 \text{ V}) / (2^{16}) = 76.29 \mu\text{V}$
16. True
17. (a) Input (b) Output (c) Input (d) Input (e) Input (f) Input
(g) Output
18. While pins A, B, and C determine the analog input to be converted, ALE is the signal that latches the address on A, B, and C. It tells the ADC that the data on A, B, and C is valid and orders the ADC to latch this data. Channel 5 (= IN5) is selected as soon as ADC receives an L-to-H edge while ABC = 101.
19. (a) 5.12 V since $256 \times 20 \text{ mV} = 5.1 \text{ V}$.
(b) 1.28 V since $255 \times 5 \text{ mV} = 1.28 \text{ V}$.
(c) 2.56 V
(d) 3.84 V
(e) 0.512 V
(f) 6.40 V
20. (a) 5 mV
(b) 3.9 mV
(c) 2.5 mV

21. True.
22. CS = Low, WR = a L-to-H transition
23. CS = Low (activated), WR = High (inactive), RD = L-to-H transition
24. The output data is put on DB0-DB7, and then a H-to-L transition on INTR announces the end of conversion.
25. The old data gets lost.
26. Output. An H-to-L transition on INTR announces the end of conversion.
27. (a) $1.28\text{V} / 256 = 5 \text{ mV}$
(b) $1 / 256 = 3.91 \text{ mV}$
(c) $1.9 / 256 = 7.42 \text{ mV}$
28. $20 \text{ mV} \times 256 = 5.1 \text{ V}$
29. 1.28 V
30. The corresponding data that selects the desired channel should be on MA0-MA4, and then a pulse should be sent on the WR signal while CS is Low (activated). The address shown on MA0-MA4 is latched into the ADC on the L-to-H transition on WR.
31. $255 \times 19.53\text{mV} = 4.98 \text{ V}$
32. Step Size = $1.28 / 256 = 5 \text{ mV}$
(a) $255 \times 5 \text{ mV} = 1.275\text{V}$
(b) $153 \times 5 \text{ mV} = 0.765$
(c) $108 \times 5\text{mV} = 0.54$
33. True.
34. (a) Input (b) Output (c) Input (d) Input (e) Input (f) Input
(g) Output
35. Low.
36. Low.
37. CHAN2 $\rightarrow 10101110 = \text{AEh}$
CHAN5 $\rightarrow 11011110 = \text{DEh}$
CHAN7 $\rightarrow 11111110 = \text{FEh}$
38. (a) $1.28\text{V} / 256 = 5 \text{ mV}$
(b) $1 \text{ V} / 256 = 3.91 \text{ mV}$
(c) $1.9 / 256 = 7.42 \text{ mV}$
39. The analog channel is selected through the control byte that should be sent to MAX1112.
40. $5 \text{ mV} \times 256 = 1.28 \text{ V}$
41. Step size = 5 mV
(a) $255 \times 5 \text{ mV} = 1.275 \text{ V}$
(b) $153 \times 5 \text{ mV} = 0.765 \text{ V}$
(c) $108 \times 5 \text{ mV} = 0.54 \text{ V}$
42. positive-edge
43. negative-edge
44. 1 V
45. 8
46. 9
47. As soon as the last bit of the control byte (PD0) is sent to MAX1112.
48. End-of-conversion is indicated by SSTRB going high.
49. We get 4.096V for full-scale, and that leads to a step size of 16 mV.
50. COM is the reference.

Section 13.2: DAC Interfacing

- 51. True.
- 52. (a) 256 (b) 1024 (c) 4096
- 53. $2 \text{ mA} \times (255/256) = 1.99 \text{ mA}$
- 54. (a) 1.195 mA since $10011001\text{B} = 153$ and $(153 \times 2 \text{ mA})/256 = 1.195 \text{ mA}$
(b) 1.594 mA since $11001100\text{B} = 204$ and $(204 \times 2 \text{ mA})/256 = 1.594 \text{ mA}$
(c) 1.859 mA since $11101110\text{B} = 238$ and $(238 \times 2 \text{ mA})/256 = 1.859 \text{ mA}$
(d) 0.2656 mA
(e) 0.0703125 mA
(f) 1.063 mA
- 55. More
- 56. All high

Section 13.3: Sensor Interfacing and Signal Conditioning

- 57. The output corresponds with the input linearly.
- 58. 10 mV
- 59. Adjusting the output of a given sensor (device) to meet the needs of the ADC chip
- 60. To maintain constant voltage so that the variation of V_{CC} source has no effect on the V_{ref} voltage. This must be done since the source voltage for V_{ref} is V_{CC} .

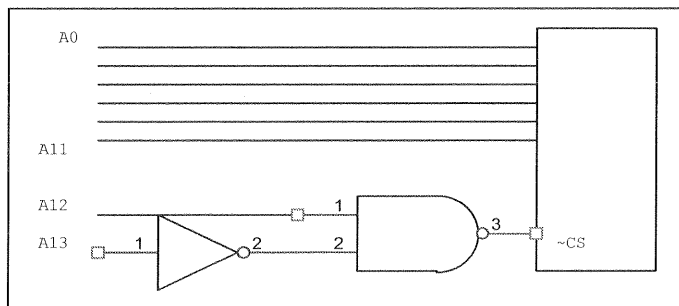
CHAPTER 14: 8051 INTERFACING TO EXTERNAL MEMORY

Section 14.1: Semiconductor Memory

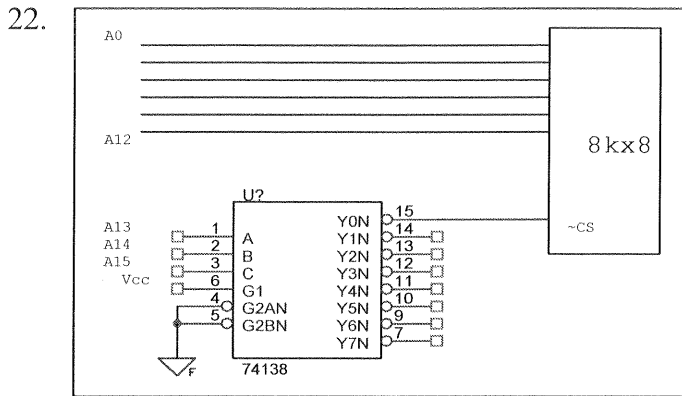
1. The storage of the chip is measured in Megabits while the Computer memory is measured in Megabytes.
2. True, the more address lines the more memory locations.
3. True, the memory location size is fixed.
4. True, the more data lines the more memory locations
5. True
6. access time
7. True
8. electrically erasable
9. True
10. DRAM
11. SRAM
12. DRAM and SRAM
13. (c)
14. (c)
15. (a) 32Kx8, 256K (f) 8Kx1, 8K
(b) 8Kx8, 64K (g) 4Kx8, 32K
(c) 4Kx8, 32K (h) 2Kx8, 16K
(d) 8Kx8, 64K (i) 256Kx4, 1M
(e) 4Mx1, 4M (j) 64Kx8, 512K
16. (a) 128K 14 8 (f) 256K 8 4
(b) 256K 15 8 (g) 8M 20 8
(c) 512K 16 8 (h) 16M 11 4
(d) 2M 18 8 (i) 512K 16 8
(e) 512K 16 8
17. 69h
18. (a) Correct
(b) Error, it should be 75h.

Section 14.2: Memory Address Decoding

19. 4000h - 7FFFh
- 20.

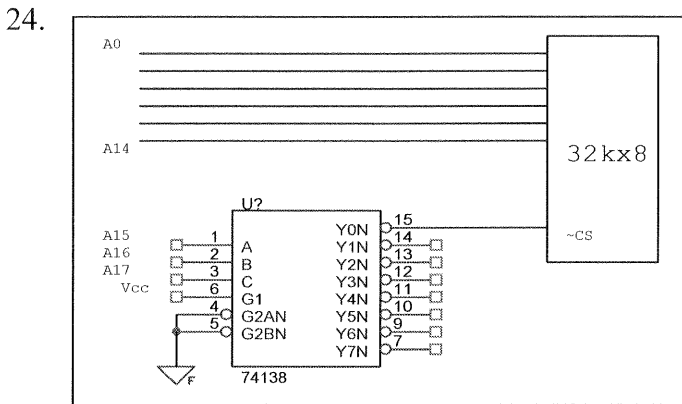


21. 8000h - 8FFFh, B000h - BFFFh, E000 - EFFFh



Each controls 8K bytes block

23. 6000h - 7FFFh, C000h - DFFFh, E000h - FFFFh



Each controls 32K bytes block

25. 4000h - 7FFFh, 8000h - BFFFh, C000h - FFFFh

Section 14.3: 8031/51 Interfacing with External ROM

26. 3FFFh

27. 0000h

28. 17, 16; 3; P3.6 and P3.7

29. 64

30. True

31. True

32. (c)

33. (a), (b), (d)

34. Port 0

35. Port 2

36. Port 0

37. ALE = 0 is for the address, while ALE = 1 is for the data.

38. 17, 6; PSEN is pin 29 and does not belong to any port.

39. (c)
40. 0000h, external ROM
41. The 8051 starts to access the on-chip ROM at address 0000 and goes on to the last location of the on-chip ROM which is 3FFFh and then goes to the external ROM for more code if it is not stopped.
42. True
43. PSEN is used for all external program fetches.

Section 14.4: 8051 Data Memory Space

44. RD and WR pins are used to access external data.
45. 64
46. (a)
47. All are active low .
48. True
49. MOVC is used to get data from code space, while MOVX is used to get data in the data space of the microcontroller.

50.

```
RAMDATA EQU 60h
ROMDATA EQU 2000h
COUNT EQU 20

MOV DPTR, #ROMDATA
MOV R2, #COUNT
MOV R1, #RAMDATA
AGAIN: MOVX A, @DPTR
MOV @R1, A
INC DPTR
INC R1
DJNZ R2, AGAIN
```

51.

```
RAMDATA EQU 40h
ERAMDATA EQU 6000h
COUNT EQU 30

MOV DPTR, #ERAMDATA
MOV R2, #COUNT
MOV R1, #RAMDATA
AGAIN: MOV A, @R1
MOVX @DPTR, A
INC DPTR
INC R1
DJNZ R2, AGAIN
```

52.

```

        ACALL ROM_IN
        ACALL RAM_OUT
        SJMP $
ROM_IN:
        RAMDATA EQU 60h
        ROMDATA EQU 3000h
        COUNT    EQU 50

        MOV DPTR, #ROMDATA
        MOV R2, #COUNT
        MOV R1, #RAMDATA
AGAIN:  MOVX A, @DPTR
        MOV @R1, A
        INC DPTR
        INC R1
        DJNZ R2, AGAIN
        RET
;-----
RAM_OUT:
        RAMDATA EQU 60h
        ERAMDATA EQU 8000h
        COUNT    EQU 50

        MOV DPTR, #ERAMDATA
        MOV R2, #COUNT
        MOV R1, #RAMDATA
AGA:    MOV A, @R1
        MOVX @DPTR, A
        INC DPTR
        INC R1
        DJNZ R2, AGA
        RET

```

53.

```

        RAMDATA EQU 60h
        ROMDATA EQU 3000h
        COUNT    EQU 50
        ACALL ROM_IN
        ACALL RAM_OUT
        SJMP $
ROM_IN:

        MOV DPTR, #ROMDATA
        MOV R2, #COUNT
        MOV R1, #RAMDATA
AGAIN:  MOVX A, @DPTR
        MOV @R1, A
        INC DPTR
        INC R1
        DJNZ R2, AGAIN
        RET
;-----
RAM_OUT:
; Continued on next page

```

```

MOV A, 0C4h
SETB ACC.0
MOV 0C4h, A
MOV DPTR, #0000h
MOV R1, #RAMDATA
MOV R2, #COUNT
NEXT: MOV A, @R1
      MOVX @DPTR, A
      INC R1
      INC DPTR
      DJNZ R2, NEXT
      RET

```

54.

```

MOV A, 0C4h
SETB ACC.0
MOV 0C4h, A
MOV DPTR, #0200h
MOV R1, #50
NEXT: MOV A, @DPTR
      MOV P1, A
      INC DPTR
      ACALL DELAY
      DJNZ R1, NEXT
      SJMP $
DELAY:
      ...

```

- 55. C4h is the address. 80h is the content upon reset.
- 56. DS89C4x0 family.
- 57. blocked
- 58. 0000h – FFFFh
- 59. 0400h – FFFFh

Section 14.5: Accessing External Data Memory in 8051 C

60.

```

#include <reg51.h>
#include <absacc.h>

void main(void)
{
    unsigned char cnt;
    unsigned char x;
    unsigned char myarray[20];
    for(cnt=0; cnt<20; cnt++)
    {
        x=XBYTE[0x2000+cnt];
        myarray[cnt]=x;
    }
}

```

61.

```
#include <reg51.h>
#include <absacc.h>

void main(void)
{
    unsigned char cnt;
    unsigned char x;
    unsigned char myarray[30]="012345678901234567890123456789";
    for(cnt=0;cnt<30;cnt++)
    {
        x = myarray[cnt];
        XBYTE[0x6000+cnt] = x;
    }
}
```

62.

```
#include <reg51.h>
#include <absacc.h>

void main(void)
{
    unsigned char cnt;
    unsigned char x;
    for(cnt=0;cnt<50;cnt++)
    {
        x=XBYTE[0x3000+cnt];
        XBYTE[0x8000+cnt]=x;
    }
}
```

63.

```
#include <reg51.h>
#include <absacc.h>
sfr PMRREG = 0xC4;
void main(void)
{
    unsigned char cnt;
    unsigned char x;
    PMRREG = PMRREG | 0x01;
    for(cnt=0;cnt<50;cnt++)
    {
        x=XBYTE[0x3000+cnt];
        XBYTE[0x0000+cnt]=x;
    }
}
```


64.

```
#include <reg51.h>
#include <absacc.h>
sfr PMRREG = 0xC4;

void MSDelay(unsigned int itime);

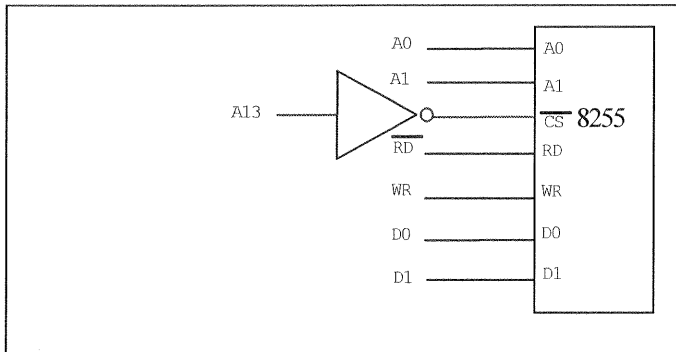
void main(void)
{
    unsigned char cnt;
    unsigned char x;
    PMRREG = PMRREG | 0x01;
    for(cnt=0;cnt<100;cnt++)
    {
        x=XBYTE[0x200+cnt];
        P1=x;
        MSDelay(1000);
    }
}

void MSDelay(unsigned int itime)
{
    unsigned int i,j;
    for(i=0;i<itime;i++)
        for(j=0;j<1275;j++);
}
```

CHAPTER 15: 8051 INTERFACING WITH THE 8255

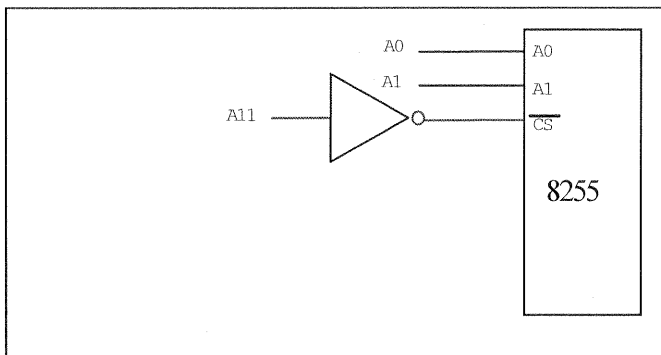
Section 15.1: Programming the 8255

1. 9Bh
2. 80h
3. 90h
4. 8Bh
5. 92h
- 6.



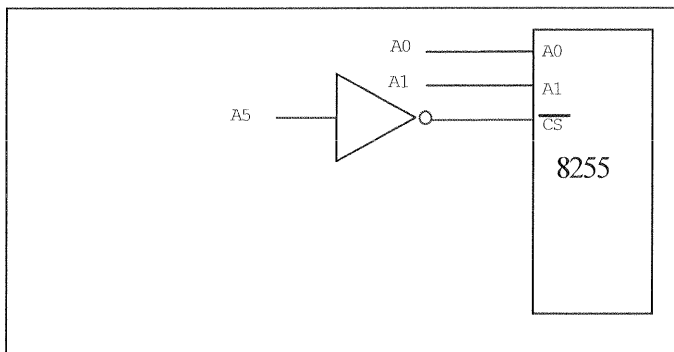
```
...  
MOV A, #82H  
MOV DPTR, #2003  
MOVX @DPTR, A  
MOV DPTR, #2001  
MOVX A, @DPTR  
MOV DPTR, #2000  
MOVX @DPTR, A  
MOV DPTR, #2002  
MOVX @DPTR, A  
...
```

7.



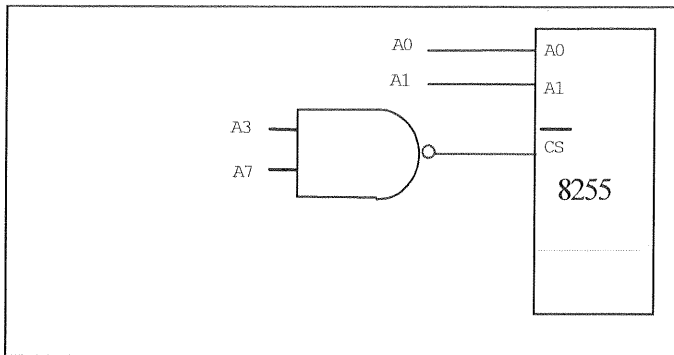
```
...  
MOV A, #89H  
MOV DPTR, #803  
MOVX @DPTR, A  
MOV DPTR, #802  
MOVX A, @DPTR  
MOV DPTR, #801  
MOVX @DPTR, A  
MOV DPTR, #802  
MOVX @DPTR, A  
...
```

8.



```
...  
MOV A, #82h  
MOV R0, #23h  
MOVX @R0, A  
MOV R0, #21h  
MOVX A, @R0  
MOV R0, #20h  
MOVX @R0, A  
MOV R0, #22h  
MOVX R0, A  
...
```

9.

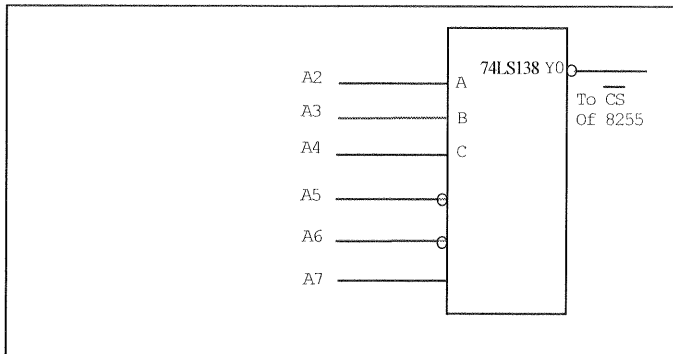


```

...
MOV A, #89h
MOV R0, #8Bh
MOVX @R0, A
MOV R0, #8Ah
MOVX A, @R0
MOV R0, #88h
MOVX @R0, A
MOV R0, #89h
MOVX R0, A
...

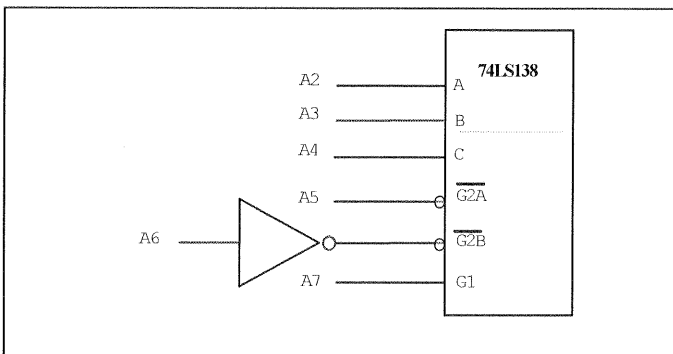
```

10.



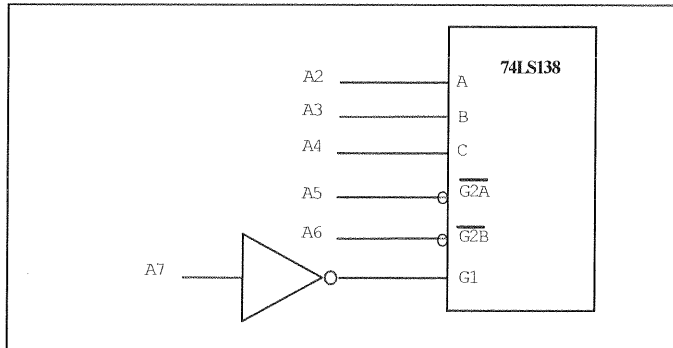
11. 84, 88, 94h

12.

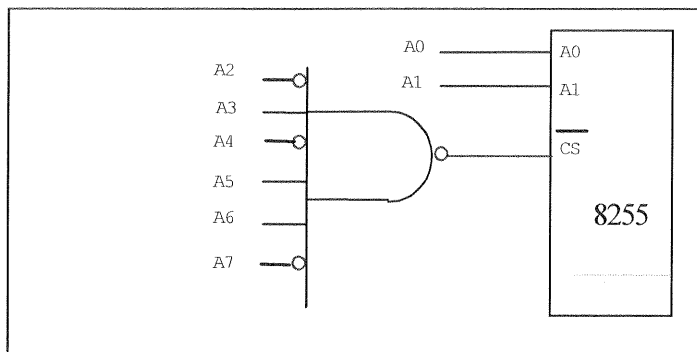


13. CCh, D4h, DCh

14.



15. 04h, 08h, 1Ch
16. 24 pins, inputs and outputs
17. sends data to and from the microcontroller
18. False, they are not bit addressable.
19. True, all can be programmed for simple I/O.
20. False, we have to use the entire port for input or output.
- 21.



22. None can be used for Port A, because the last 2 bits must be 0.
23. Port A and lower Port C are inputs and the rest are outputs.
24. 92h
- 25.

```

...
MOV    R0, #PA
BACK:  MOVX  A, @R0
        CJNE  A, #100, BACK
        MOV   R3, A
        INC   R0
        MOVX  A, #0AAH
        MOVX  @R0, A
        CPL   A
        INC   R0
        MOVX  @R0, A
...

```

26.

```

#include<reg51.h>
#include<absacc.h>
void main()
{
    unsigned char x;

    XBYTE[0x4003]= 0x90;  //PA=in, PB=C=out
    while(1)
    {
        x=XBYTE[0x4000];
        if(x==100)
        {
            XBYTE[0x4001]= 0xAA;
            XBYTE[0x4002]= 0x55;
        }
    }
}

```

27.

```
#include<reg51.h>
#include<absacc.h>

void main()
{
    unsigned char x;

    XBYTE[0x4003]= 0x90;  //PA=in, PB=PC=out
    while(1)
    {
        x=XBYTE[0x4000];
        XBYTE[0x4001]= x;
        XBYTE[0x4002]= x;
    }
}
```

28.

```
#include<reg51.h>
#include<absacc.h>

void main()
{
    unsigned char x;
    XBYTE[0x4003]= 0x89;  //PA=PB=out, PC=in
    while(1)
    {
        x=XBYTE[0x4002];
        XBYTE[0x4000]= x;
        XBYTE[0x4001]= x;
    }
}
```

CHAPTER 16: DS12887 RTC INTERFACING AND PROGRAMMING

Section 16.1: DS12887 RTC Interfacing

1. 24
2. 24, 12
3. 8
4. False
5. False
6. 2100
7. DS – Data strobe, AS – Address strobe, and MOT – Motorola select
8. Input
9. High, low
10. Continues to operate
11. Data strobe, input
12. Vcc
13. Input, RD
14. Input, ALE
15. AS (Address select)
16. Output
17. Output
18. Input
19. 128
20. Unaffected
21. Unaffected
22. When Vcc drops below 3V
23. 0-0D
24. 0A-0C Hex
25. Register B, bit 1
26. Register B, bit 0
27. 09
28. 7F
- 29.

```
...  
MOV R0,#4           ;point to HR loc  
MOVX A,@R0          ;read hours  
ACALL DISPLAY  
MOV A,#20H          ;send out SPACE  
ACALL SERIAL  
MOV R0,#2           ;point to minute loc  
MOVX A,@R0          ;read minute  
ACALL DISPLAY  
...
```

30.

```
...
MOV R0,#09      ;point to year loc
MOVX A,@R0      ;read year
ACALL DISPLAY
...
```

31.

```
...
MOV R0,#4       ;point to HR loc
MOVX A,@R0      ;read hours
MOV P1, A
MOV R0,#2       ;point to minute loc
MOVX A,@R0      ;read minute
MOV P2, A
...
```

32.

```
...
MOV R0,#0       ;point to seconds address
MOV A,#05H      ;seconds=05H (BCD numbers need H)
MOVX @R0,A      ;set seconds
MOV R0,#02      ;point to minutes address
MOV A,#15H      ;minutes=15
MOVX @R0,A      ;set minutes
MOV R0,#04      ;point to hours address
MOV A,#09H      ;hours=09
MOVX @R0,A      ;set hours
MOV R0,#11      ;Reg B address
MOV A,#03       ;D7=0 of reg B to allow update
MOVX @R0,A      ;send it to reg B
...
```

33.

```
...
MOV R0,#0       ;point to seconds address
MOV A,#19H      ;seconds=19H (BCD numbers need H)
MOVX @R0,A      ;set seconds
MOV R0,#02      ;point to minutes address
MOV A,#47H      ;minutes=47
MOVX @R0,A      ;set minutes
MOV R0,#04      ;point to hours address
MOV A,#22H      ;hours=22
MOVX @R0,A      ;set hours
MOV R0,#11      ;Reg B address
MOV A,#03       ;D7=0 of reg B to allow update
MOVX @R0,A      ;send it to reg B
...
```

34.

```
...
MOV R0,#07          ;load pointer for DAY OF MONTH
MOV A,#14H          ; DAY=14H (BCD numbers need H)
MOVX @R0,A          ;set DAY OF MONTH
ACALL DELAY         ;
MOV R0,#08          ;point to MONTH
MOV A,#05H          ;05=MAY
MOVX @R0,A          ;set MONTH
ACALL DELAY         ;
MOV R0,#09          ;point to YEAR address
MOV A,#09           ;YEAR=09 FOR 2009
MOVX @R0,A          ;set YEAR to 2009
ACALL DELAY
MOV R0,#11          ;Reg B address
MOV A,#03           ;D7=0 of reg B to allow update
MOVX @R0,A
...
```

35. The last Sunday

Section 16.2: DS12887 RTC Programming in C

36.

```
//written and tested by Chris Byrd
#include <reg51.h>
#include <absacc.h>
void bcdconv(unsigned char);
void serial(unsigned char);
void Delay(unsigned int);
void main(void)
{
    unsigned char hr, min, sec;
    TMOD=0x20;
    TH1=0xFD;          //9600 baud rate
    SCON=0x50;
    TR1=1;

    Delay(200);        //RTC needs 200 ms upon power-up
    XBYTE[10]=0x20;    //turn on oscillator
    XBYTE[11]=0x81;    //BCD, 12 hrs, daylight savings
    XBYTE[11]=0x01;    //allow update

    while(1)
    {
        hr=XBYTE[4];   //get hour
        bcdconv(hr);   //convert and display
        serial(':');   //send out : to separate
        min=XBYTE[2];  //get minute
        bcdconv(min);  //convert and display
        serial(':');   //send out : to separate
        sec=XBYTE[0];  //get second
        bcdconv(sec);  //convert and display
        if(hr>0x12)    //calculate to display PM
        {
            //if hour greater than 12
            serial(' ');
            serial('P');
            serial('.');
            serial('M');
            serial('.');
        }
    }
}
```


36. continued

```

        serial('.');
    }
    else //otherwise display AM
    {
        serial(' ');
        serial('A');
        serial('.');
        serial('M');
        serial('.');
    }
    serial(0x0D); //send out CR
    serial(0x0A); //send out Line Feed
}
}
// convert BCD to ASCII and send it to serial
void bcdconv(unsigned char mybyte) //see Chapter 7
{
    unsigned char x, y;
    x=mybyte&0x0F;
    x=x|0x30;
    y=mybyte&0xF0;
    y=y>>4;
    y=y|0x30;
    if(y==0x38) //convert 81-92 values (PM hours)
        y=0x30;
    else if(y==0x39) //to 01-12 for displaying
        y=0x31;
    serial(y);
    serial(x);
}
// send out one char serially
void serial(unsigned char x)
{
    SBUF=x;
    while(TI==0);
    TI=0;
}

void Delay(unsigned int itime)
{
    unsigned int i, j;
    for(i=0;i<itime;i++)
        for(j=0;j<1275;j++);
}

```

37.

```

//written and tested by Chris Byrd
void main(void)
{
    unsigned char yr_lo, yr_hi;

    Delay(200); //RTC needs 200 ms upon power-up
    XBYTE[10]=0x20; //turn on oscillator
    XBYTE[11]=0x81; //BCD, 12 hrs, daylight savings
    XBYTE[11]=0x01; //allow update

    yr_hi=0x20; //century
    yr_lo=XBYTE[9]; //get year
    P2=yr_hi; //send to ports 1 and 2
    P1=yr_lo;
}

```

38.

```
//written and tested by Chris Byrd
void main(void)
{
    unsigned char hr, min;

    Delay(200);           //RTC needs 200 ms upon power-up
    XBYTE[10]=0x20;       //turn on oscillator
    XBYTE[11]=0x87;       //binary, 24 hrs, daylight savings
    XBYTE[11]=0x07;       //allow update

    hr=XBYTE[4];
    min=XBYTE[2];
    P2=hr;
    P1=min;
}
```

39.

```
//written and tested by Chris Byrd
void main(void)
{
    Delay(200);           //RTC needs 200 ms upon power-up
    XBYTE[10]=0x20;       //turn on oscillator
    XBYTE[11]=0x81;       //BCD, 12 hrs, daylight savings
    XBYTE[0]=0x05;        //SECOND=05h for BCD
    XBYTE[2]=0x15;        //MINUTE=15h for BCD
    XBYTE[4]=0x89;        //HOUR=89h for BCD/RTC for 9pm
    XBYTE[11]=0x01;       //allow update
}
```

40.

```
//written and tested by Chris Byrd
void main(void)
{
    Delay(200);           //RTC needs 200 ms upon power-up
    XBYTE[10]=0x20;       //turn on oscillator
    XBYTE[11]=0x83;       //BCD, 24 hrs, daylight savings
    XBYTE[0]=0x19;        //SECOND=19h for BCD
    XBYTE[2]=0x47;        //MINUTE=47h for BCD
    XBYTE[4]=0x22;        //HOUR=22h for BCD
    XBYTE[11]=0x03;       //allow update
}
```

41.

```
//written and tested by Chris Byrd
void main(void)
{
    Delay(200);           //RTC needs 200 ms upon power-up
    XBYTE[10]=0x20;       //turn on oscillator
    XBYTE[11]=0x83;       //BCD, 24 hrs, daylight savings
    XBYTE[7]=0x14;        //day=14h
    XBYTE[8]=0x05;        //month=05h for May
    XBYTE[9]=0x09;        //year=09h for 2009
    XBYTE[11]=0x03;       //allow update
}
```

42. The 20 part of the year 2009 is assumed, since the RTC works for 2000-2099.

Section 16.3: ALARM, SQW, and IRQ Features of the DS12887 Chip

43. Output, low
44. Output
45. 5, set bit high
46. 6, set bit high
47. 5
48. 6
49. 2Hz
50. 8.192 kHz
51. Alarm interrupt, Periodic Pulse interrupt, Update interrupt
52. 500 milliseconds
53. 122 microseconds
54. To generate an interrupt for the 8051
55. To generate an interrupt for the 8051
56. the PIE will simply direct the PI bit of register C into the IRQ output pin.
57. the AIE will simply direct the AI bit of register C into the IRQ output pin
58. Set bit 3 high
59. Register A
60. Register A
61. Register B
62. The IRQ pin will be asserted low when all 3 bytes of the real time are the same as the alarm bytes.
63. Directs the PI bit of register C into the IRQ pin
64.

```
...  
MOV R0,#10      ;R0 = 0AH, reg A address  
MOV A,#27H      ;turn on osc., 512Hz SQW  
MOVX @R0,A      ;send it to Reg A of DS12887  
MOV R0,#11      ;R0 = 0BH, Reg B address  
MOVX A,@R0      ;get reg B of DS12887 to ACC  
ACALL DELAY     ;need delay for fast 8051  
SETB ACC.3      ;let 512Hz come out  
MOVX @R0,A      ;send it back to reg B
```

- 65.
- ```
...
MOV R0,#10 ;R0 = 0AH, reg A address
MOV A,#2AH ;turn on osc., 64Hz SQW
MOVX @R0,A ;send it to Reg A of DS12887
MOV R0,#11 ;R0 = 0BH, Reg B address
MOVX A,@R0 ;get reg B of DS12887 to ACC
ACALL DELAY ;need delay for fast 8051
SETB ACC.3 ;let 64Hz come out
MOVX @R0,A ;send it back to reg B
```

## **CHAPTER 17: MOTOR CONTROL: RELAY, PWM, DC, AND STEPPER MOTOR**

### **Section 17.1: Relays and Optoisolators**

1. False
2. True
3. Switching response time, life expectancy, low input current and small packaging
4. False
5. 4.167 mA
6. Motor control, Modem
7. It can be directly connected to the microcontroller
8. EM
9. True
10. False

### **Section 17.2: Stepper Motor Interfacing**

11. 4 degree
12. 48
13. 1001, 1100, 0110, 0011
14. 0110, 0011, 1001, 1100
15. 1001, 0011, 0110, 1100
16. 1100, 1001, 0011, 0110
17. Supplies the current required for the motor. No, it only supplies up to 500 mA.
18. b
19. Increasing or decreasing the delay between the steps changes the speed of the motor
20. Shorten the time delay between steps

### **Section 17.3: DC Motor Interfacing and PWM**

21. Stepper
22. True
23. False
24. No-load
25. DC motors can be controlled using PWM
26. Steppers are used for positioning
27. True
28. Pulse width modulation, controls the speed of the motor.
29. Keep the PWM the same
30. Protects the microcontroller from the motor's back EMF.